

COURS · NOTEBOOKS ET EXERCICES SUR GITHUB

Data Science & IA · 13 séances

Comprendre les données et l'IA, en construisant. Python, données, machine learning, puis l'IA générative.

<https://cours-data-science.vercel.app>

Sommaire

SÉANCE 00 · Faire connaissance

SÉANCE 01 · Introduction à l'intelligence artificielle

SÉANCE 02 · Python pour la data

SÉANCE 03 · SQL et Git

SÉANCE 04 · Collecter et nettoyer

SÉANCE 05 · Analyser et raconter

SÉANCE 06 · Premier modèle

SÉANCE 07 · Kaggle Titanic 1/2

SÉANCE 08 · Kaggle Titanic 2/2

SÉANCE 09 · Comment fonctionne un LLM

SÉANCE 10 · Parler à un LLM par le code

SÉANCE 11 · Le RAG

SÉANCE 12 · Agents et projet final

Faire connaissance

On fait connaissance, on regarde ce que chacun a déjà touché, on installe les trois comptes, et Python répond pour la première fois dans Colab.

Les trente premières minutes

Six écrans, à faire défiler ensemble : qui je suis, une affirmation à débattre, c'est quoi l'IA pour toi, le quiz « est-ce de l'IA ? », ce que tu as déjà fait, et le parcours en quatre blocs. Tout ce que tu tapes reste dans cette page ; à la fin, l'écran 6 copie le récap de vos réponses en un clic.

Ensuite, le notebook

Le reste de la séance se passe dans Colab, ensemble. Rien n'est renvoyé à la maison : ce qui n'est pas fini reprend au créneau suivant.

Python répond ? puis le **diagnostic en 6 mini-exercices** (15 min) : variables, boucles, fonctions, listes. Ce n'est pas noté, c'est pour savoir d'où on part.

Dictionnaires, puis brainstorm de projets (10 min) : chacun écrit 3 idées dans une liste Python. L'« outil rêvé » de l'écran 5 est la première.

Les trois comptes (10 min) : Google Colab, Kaggle, GitHub, et la création du premier dépôt `mon-portfolio-ia`. C'est la seule étape qui doit être finie avant de partir.

Le jeu du nombre mystère (10 min) : d'abord des coups prédéfinis, puis la stratégie du milieu.

Dans Colab, pense à *Fichier → Enregistrer une copie dans Drive* : sans ça, tout disparaît à la fermeture de l'onglet.

Entre deux séances

Le carnet d'exercices `00_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées. C'est le seul travail personnel du parcours : la leçon, elle, se fait toujours ensemble. Il s'ouvre par un quiz de trois questions, celles qu'on se pose à voix haute au début de la séance suivante.

Introduction à l'intelligence artificielle

D'où vient l'IA, pourquoi elle explose maintenant, ce que recouvrent les mots IA, machine learning et deep learning, et à quoi ressemble un réseau de neurones.

À LA FIN DE LA SÉANCE

Tu sauras dire pourquoi 2012 est la vraie rupture de la frise, et pas 1957 ni 2022.

Tu sauras placer un exemple dans le bon cercle, IA, machine learning ou deep learning.

Tu sauras reconnaître une classification d'une régression à la forme de la réponse.

Une frise, de 1950 à aujourd'hui

L'IA n'est pas née avec ChatGPT : l'idée a soixante-quinze ans. Fais défiler. Quand des sources sérieuses ne s'accordent pas sur un chiffre, la carte le dit.

LES FONDATIONS • 1950 – 1969

octobre 1950 • Le test de Turing : Dans la revue Mind, Alan Turing propose le « jeu de l'imitation » : une machine réussit si un juge, par écrit, ne parvient pas à la distinguer d'un humain.

été 1956 • La conférence de Dartmouth : Un atelier d'été réunit McCarthy, Minsky, Rochester et Shannon ; leur proposition de 1955 emploie pour la première fois l'expression « intelligence artificielle ».

1957 – 1958 • Le perceptron de Rosenblatt : Frank Rosenblatt décrit le perceptron, un premier neurone artificiel qui apprend à reconnaître des formes ; le New York Times y voit l'embryon d'une machine consciente.

janvier 1966 • ELIZA : Au MIT, Joseph Weizenbaum publie ELIZA, un programme qui imite un psychologue en reformulant les phrases de l'utilisateur grâce à quelques règles simples.

LES HIVERS • 1970 – 1993

1973 • Le rapport Lighthill et le premier hiver : Commandé par le Science Research Council britannique, le rapport Lighthill juge les promesses de l'IA non tenues et entraîne l'arrêt des financements dans la plupart des universités du Royaume-Uni.

1980 – 1993 • Essor puis chute des systèmes experts : XCON, en service chez DEC dès 1980, économise 25 millions de dollars par an ; en 1987 le marché des machines LISP s'effondre et un second hiver commence.

9 octobre 1986 • La rétropropagation popularisée : Rumelhart, Hinton et Williams publient dans Nature l'article qui popularise la rétropropagation, la méthode d'entraînement des réseaux de neurones à plusieurs couches encore utilisée aujourd'hui.

LA RENAISSANCE • 1994 – 2011

11 mai 1997 • Deep Blue bat Kasparov : À New York, l'ordinateur Deep Blue d'IBM bat le champion du monde d'échecs Garry Kasparov 3,5 à 2,5, un an après avoir perdu le premier match 4 à 2.

2009 · ImageNet : L'équipe de Fei-Fei Li présente ImageNet, une base de plus de 14 millions d'images annotées à la main, qui sert de référence aux concours de reconnaissance visuelle dès 2010.

LE DEEP LEARNING · 2012 - 2016

30 septembre 2012 · AlexNet : Le réseau de neurones AlexNet, entraîné sur deux cartes graphiques, remporte le concours ImageNet avec 15,3 % d'erreur, plus de 10 points devant le deuxième.

16 janvier 2013 · word2vec : Chez Google, Tomáš Mikolov et ses collègues publient word2vec, qui représente chaque mot par une liste de nombres telle que « frère » – « homme » + « femme » donne « sœur ».

9 – 15 mars 2016 · AlphaGo bat Lee Sedol : À Séoul, AlphaGo de DeepMind bat le champion de go Lee Sedol 4 à 1, un exploit que la plupart des experts n'attendaient pas avant plusieurs années.

LES TRANSFORMERS · 2017 - 2022

12 juin 2017 · « Attention is all you need » : Huit chercheurs, alors chez Google (dont un étudiant de Toronto en stage), décrivent le Transformer, une architecture fondée uniquement sur le mécanisme d'attention, qui devient la base de presque tous les grands modèles de langage.

11 octobre 2018 · BERT : Google publie BERT, un modèle qui lit un texte dans les deux sens à la fois et devient la référence pour de nombreuses tâches de compréhension du langage.

14 février 2019 · GPT-2 : OpenAI annonce GPT-2, 1,5 milliard de paramètres, mais refuse d'abord de publier le modèle complet par crainte d'usages malveillants ; il est diffusé le 5 novembre 2019.

28 mai 2020 · GPT-3 : OpenAI présente GPT-3, 175 milliards de paramètres, dix fois plus que tout modèle précédent, capable d'accomplir une tâche à partir de quelques exemples donnés dans la consigne.

novembre 2020 · AlphaFold 2 : AlphaFold 2 de DeepMind remporte le concours CASP14 en prédisant la forme des protéines ; l'article paraît dans Nature le 15 juillet 2021 avec le code en accès libre.

avril – août 2022 · L'année des images générées : DALL·E 2 (6 avril), Midjourney (bêta ouverte le 12 juillet) puis Stable Diffusion (22 août, poids publics) permettent à chacun de créer une image à partir d'une phrase.

30 novembre 2022 · ChatGPT : OpenAI ouvre ChatGPT, fondé sur GPT-3.5 : un million d'utilisateurs en cinq jours, cent millions en deux mois, et l'IA générative entre dans le grand public.

LE BOOM · 2023 - AUJOURD'HUI

24 février 2023 · LLaMA et les modèles ouverts : Meta réserve LLaMA aux chercheurs sous licence non commerciale ; les poids fuient le 3 mars, puis Llama 2 (18 juillet) autorise l'usage commercial et lance la vague des modèles ouverts.

14 mars 2023 · GPT-4 (et Claude) : OpenAI lance GPT-4, qui accepte aussi des images et se classe parmi les 10 % meilleurs à un examen du barreau simulé ; le même jour, Anthropic présente Claude.

mars – novembre 2023 · L'année des alertes : Une lettre ouverte demande une pause de six mois (22 mars), Geoffrey Hinton quitte Google pour alerter librement (1er mai), et 28 pays signent la déclaration de Bletchley (1er-2 novembre).

6 décembre 2023 · Gemini : Google annonce Gemini 1.0, conçu d'emblée pour traiter texte, code, audio, image et vidéo ; sa version Ultra est présentée comme la première à dépasser les experts humains sur le test MMLU.

13 mars 2024 · L'AI Act européen : Le Parlement européen adopte l'AI Act par 523 voix contre 46 ; en vigueur le 1er août 2024, il s'applique par étapes : interdictions en février 2025, modèles généraux en août 2025.

12 septembre 2024 · o1 et les modèles de raisonnement : OpenAI lance o1-preview, un modèle qui « réfléchit » avant de répondre : il résout 83 % des problèmes d'un examen qualificatif d'olympiade de mathématiques, contre 13 % pour GPT-4o.

8 et 9 octobre 2024 · Deux prix Nobel pour l'IA : John Hopfield et Geoffrey Hinton reçoivent le Nobel de physique pour les réseaux de neurones ; le lendemain, David Baker, Demis Hassabis et John Jumper obtiennent celui de chimie, notamment pour AlphaFold.

20 – 27 janvier 2025 · DeepSeek R1 secoue les marchés : La start-up chinoise DeepSeek publie R1, un modèle de raisonnement ouvert et peu coûteux ; le 27 janvier, Nvidia perd 589 milliards de dollars en Bourse, un record historique.

janvier – mars 2025 · L'année des agents : OpenAI lance Operator (23 janvier), un agent qui navigue sur le web à la place de l'utilisateur ; Manus (6 mars) et d'autres popularisent des agents qui planifient et exécutent des tâches.

janvier – avril 2025 · Centres de données et énergie : Stargate promet jusqu'à 500 milliards de dollars de centres de données aux États-Unis (21 janvier) ; l'AIE prévoit que leur consommation électrique mondiale passe de 415 TWh en 2024 à 945 TWh en 2030.

7 août 2025 · GPT-5 : OpenAI déploie GPT-5 à tous les utilisateurs de ChatGPT, alors 700 millions par semaine ; un « routeur » choisit entre réponse rapide et réflexion longue, et l'accueil est mitigé.

29 octobre 2025 · Nvidia vaut 5 000 milliards : Nvidia devient la première entreprise valorisée 5 000 milliards de dollars, trois mois après le cap des 4 000 ; ChatGPT compte alors 800 millions d'utilisateurs hebdomadaires.

9 juin 2026 · Claude Fable 5 et Mythos 5 : Anthropic ouvre au public Claude Fable 5, version bridée de son modèle Mythos : les requêtes sensibles (cybersécurité, biologie, chimie) sont détournées vers un modèle moins puissant.

27 juillet 2026 · L'AI Act ajusté : l'omnibus numérique : Le règlement « omnibus » entre en vigueur et repousse les obligations des systèmes à haut risque au 2 décembre 2027 ; les règles de transparence s'appliquent bien le 2 août 2026.

28 juillet 2026 · Les employés de l'IA demandent un frein : Plus de 1 100 salariés d'OpenAI, Anthropic, Google et Meta, dont Dario Amodei, demandent aux États-Unis des outils pour pouvoir ralentir le développement automatisé de l'IA si nécessaire.

3 septembre 2026 · GPT-6 Astra : OpenAI dévoile GPT-6 Astra, entraîné sur plus de 100 000 GPU au site Stargate du Texas ; son président Greg Brockman parle d'entrée dans « l'ère de l'AGI », ce que des experts contestent.

01

Hivers

2

Deux fois, les promesses n'ont pas été tenues et les financements ont disparu. Ça peut arriver de nouveau.

[pour aller plus loin](#)

Premier hiver, vers 1974–1980 : le rapport Lighthill (1973) juge les promesses non tenues, les financements britanniques puis américains se tarissent.

Second hiver, vers 1987–1993 : le marché des machines Lisp s'effondre en 1987, le projet japonais de cinquième génération s'achève en 1992 sans avoir tenu ses objectifs.

Lighthill report (Wikipedia) ↗ · AI winter (Wikipedia) ↗ · vérifié le 2026-09-25

02

Accélération

1

Une seule architecture depuis 2017, le transformer, derrière les modèles de texte, d'image, de son et de code.

[pour aller plus loin](#)

Un transformer lit tous les mots d'une phrase en même temps et apprend lesquels regarder pour comprendre chacun (le mécanisme d'attention), au lieu de les lire un par un.

Décrit en juin 2017 par huit chercheurs alors chez Google, dans « Attention Is All You Need ».

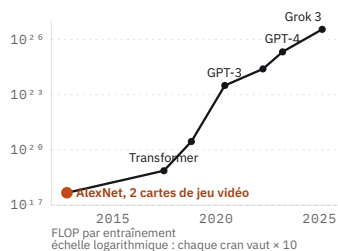
Vaswani et al., arXiv:1706.03762 (2017) ↗ · vérifié le 2026-09-25

Pourquoi maintenant

Les idées sont anciennes, les moyens sont récents : le neurone artificiel date de 1957, et il a fallu attendre 2012 pour qu'un ordinateur soit assez puissant, et les données assez nombreuses, pour qu'il serve vraiment. Trois causes à ce rattrapage, aucune ne suffit seule ; « détails et sources » dit d'où vient chaque chiffre.

CAUSE 1

La puissance de calcul



Multiplié par 4 à 5 chaque année : de 10^{17} à plus de 10^{26} opérations par entraînement en treize ans (Epoch AI).

détails et sources

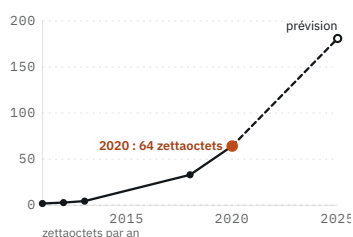
Un réseau de neurones passe son temps à faire la même multiplication sur des millions de nombres à la fois. Une carte graphique, conçue pour calculer les pixels des jeux vidéo, est faite exactement pour ça.

Les points de la courbe (Epoch AI, base des modèles notables, consultée le 25/09/2026) : AlexNet 2012, $4,7 \times 10^{17}$ FLOP ; Transformer 2017, $7,4 \times 10^{18}$; BERT-Large 2018, $2,9 \times 10^{20}$; GPT-3 2020, $3,1 \times 10^{23}$; PaLM 2022, $2,5 \times 10^{24}$; GPT-4 2023, $2,1 \times 10^{25}$; Grok 3 2025, $3,5 \times 10^{26}$.

Un processeur grand public récent, l'Intel Core Ultra 9 285K (fin 2024), a 24 cœurs. Une carte graphique GeForce RTX 5090 (2025) en a 21 760. Les deux sortes de cœurs ne se valent pas un pour un : ce qui compte, c'est le nombre d'opérations menées en même temps.

CAUSE 2

Le déluge de données



Un zettaoctet vaut mille milliards de gigaoctets ; estimations IDC successives, 2025 en pointillé car prévue en 2021.

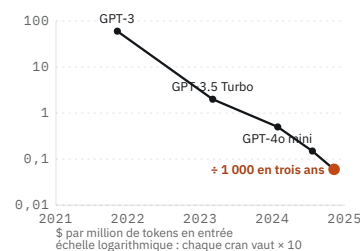
détails et sources

Un zettaoctet, c'est mille milliards de gigaoctets : de quoi remplir 8 milliards de smartphones de 128 Go. Textes, photos, vidéos, clics : c'est la matière première. Sans elle, les réseaux de neurones actuels n'auraient rien à apprendre, et c'est pour ça qu'ils ne pouvaient pas exister il y a vingt ans.

Les points de la courbe, d'après les études IDC successives : 1,8 zettaoctet en 2011 (Digital Universe 2011), 2,8 en 2012 (Digital Universe, déc. 2012), 4,4 en 2013 (Digital Universe 2014), 33 en 2018 (Data Age 2025, nov. 2018), 64,2 en 2020 et 181 prévus pour

CAUSE 3

Un coût divisé par mille



À niveau égal, le prix est divisé par dix chaque année : meilleurs algorithmes, meilleures puces (a16z, OpenAI).

détails et sources

En juin 2017, un article de huit chercheurs, alors chez Google, propose le transformer : un réseau qui lit tous les mots d'une phrase en même temps au lieu d'un par un, donc exactement ce qu'un GPU sait accélérer. Presque tous les grands modèles actuels en descendent. Et les idées continuent de s'améliorer : à puissance égale, on fait deux fois mieux tous les huit mois environ.

Les points de la courbe : GPT-3 davinci, 60 \$ le million de tokens en novembre 2021 (a16z) ; GPT-3.5 Turbo, 2 \$ le 1er mars 2023 (OpenAI) : « 10 fois moins cher » ; 0,50 \$ en entrée le 25 janvier 2024 (OpenAI) ; GPT-4o mini, 0,15 \$ en entrée le 18 juillet 2024 (OpenAI) ; Llama 3.2 3B, 0,06 \$ fin 2024 (a16z). Même niveau de test MMLU, environ 42, d'un bout à l'autre.

Vaswani et al., « Attention Is All You Need », soumis le 12 juin 2017 : huit auteurs, tous chez Google Brain ou Google Research à l'époque, l'un

AlexNet (2012), le réseau qui a lancé le deep learning moderne, a été entraîné pendant cinq à six jours sur deux cartes de jeu vidéo GTX 580.

D'après Epoch AI, le calcul consacré à l'entraînement des plus gros modèles est multiplié par 4 à 5 chaque année depuis 2010.

Le GPU que Colab prête en général gratuitement, le NVIDIA T4 (2018) : 2 560 cœurs et 16 Go de mémoire. Google ne garantit pas le modèle attribué.

Krizhevsky, Sutskever, Hinton, « ImageNet Classification with Deep Convolutional Neural Networks », NeurIPS 2012 ↗ · vérifié le 2026-09-25

aussi : Intel Core Ultra 9 285K, fiche technique Intel · GeForce RTX 5090, fiche NVIDIA · NVIDIA T4, fiche produit · Epoch AI, « Training Compute of Frontier AI Models Grows by 4-5x per Year » (mai 2024) · NVIDIA, « What's the Difference Between a CPU and a GPU? » · Google Colab, FAQ (les GPU disponibles varient) · Epoch AI, base « Notable AI models » (CSV) · Epoch AI, « Over 30 AI models have been trained at the scale of GPT-4 » (2025)

2025 (Global DataSphere, mars 2021). Les méthodes ont changé d'une édition à l'autre : c'est un ordre de grandeur, pas une série continue.

ImageNet, la base d'images qui a permis AlexNet : 14 millions d'images annotées à la main, dont 1,2 million dans 1 000 catégories pour le concours de 2012.

Llama 3 (Meta, 2024) a été entraîné sur plus de 15 000 milliards de mots-morceaux (tokens), sept fois plus que la version précédente.

IDC, « Global DataSphere Forecast 2021–2025 », mars 2021, cité par Redmond Channel Partner (le communiqué IDC n'est plus accessible) ↗ · vérifié le 2026-09-25

aussi : IDC, « The Digitization of the World » (nov. 2018) : la définition du zettaoctet · Data Center Knowledge, « Digital Universe to Add 1.8 Zettabytes in 2011 » · ImageNet, page « About » · Meta, « Introducing Meta Llama 3 » (avril 2024) · IDC, « The Digital Universe in 2020 » (déc. 2012) : 2,8 zettaoctets en 2012 · IDC/EMC, Digital Universe 2014 : 4,4 zettaoctets en 2013 (cité par SecurityWeek)

d'eux étudiant à l'université de Toronto en stage.

Ho et al. (Epoch AI, mars 2024) : le calcul requis pour un niveau donné « a été divisé par deux environ tous les 8 mois » entre 2012 et 2023, avec une large incertitude (5 à 14 mois). Le tableau de bord Epoch de février 2026 donne 7,6 mois.

Pour comparaison, le matériel progresse moins vite : la puissance des puces d'IA double tous les 2,3 ans environ (Epoch AI).

Vaswani et al., « Attention Is All You Need », arXiv:1706.03762, 12 juin 2017 ↗ · vérifié le 2026-09-25 aussi : Ho et al., « Algorithmic progress in language models », arXiv:2403.05812, mars 2024 · Epoch AI, Trends dashboard (févr. 2026) · University of Toronto, « U of T undergrad co-authors important machine learning study at Google » · a16z, « LLMflation: LLM inference cost is going down fast » (12 nov. 2024) · OpenAI, « Introducing ChatGPT and Whisper APIs » (1er mars 2023) · OpenAI, « New embedding models and API updates » (25 janv. 2024) · OpenAI, « GPT-4o mini: advancing cost-efficient intelligence » (18 juil. 2024)

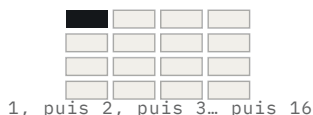
Processeur (CPU)

quelques cœurs, rapides, bons à tout



une case après l'autre

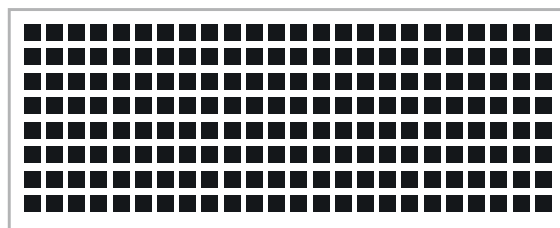
la même multiplication de matrices, 16 cases à calculer



1, puis 2, puis 3... puis 16

Carte graphique (GPU)

des milliers de cœurs, lents, tous identiques



toutes les cases en même temps

la même multiplication de matrices, 16 cases à calculer



16 d'un coup

Un réseau de neurones ne fait presque que des multiplications de matrices, toutes identiques : le GPU les fait toutes en même temps.

- POUR L'ÉLÈVE CURIEUX : POURQUOI UNE CARTE DE JEU VIDÉO

Un réseau de neurones ne fait presque qu'une seule chose : **multiplier des tableaux de nombres**. Chaque neurone additionne ses entrées pondérées par des poids ; avec des milliers de neurones par couche, ça devient une multiplication de matrices, répétée pour chaque couche, pour chaque exemple, à chaque tour d'apprentissage. Des milliards d'opérations, toutes **identiques** et **indépendantes** les unes des autres.

Ton processeur (le CPU) est fait pour l'inverse : quelques cœurs très rapides et très malins, capables d'enchaîner des tâches différentes, de prendre des décisions, de sauter d'un programme à l'autre. C'est parfait pour faire tourner un navigateur, un jeu et une messagerie en même temps. C'est un chef étoilé : il fait tout, très bien, un plat à la fois.

Une carte graphique (le GPU) a été inventée pour les jeux vidéo, où il faut calculer la couleur de millions de pixels, tous avec la même formule, soixante fois par seconde. Elle contient donc des **milliers de petits cœurs**, lents et bêtes pris un par un, mais qui font tous la même opération en même temps. C'est une cantine : mille personnes, une seule recette, tout le monde servi d'un coup.

Or « la même opération sur des millions de nombres », c'est exactement ce que demande un réseau de neurones. En 2012, trois chercheurs de Toronto ont entraîné leur réseau de reconnaissance d'images, AlexNet, pendant cinq à six jours sur deux cartes de jeu vidéo au lieu de processeurs classiques : c'est le déclic du deep learning moderne, et c'est pour ça que l'entreprise qui fabriquait ces cartes, NVIDIA, est devenue l'une des plus valorisées au monde. Dans Colab, le GPU que tu utiliseras aux séances 9 à 12 (en général un T4, une carte de 2018) vient de cette histoire.

Est-ce que ça va durer ?

La carte « Hivers » de la frise pose la question, et le marché a misé des centaines de milliards sur la réponse : quatre choses doivent tenir pour que ce pari soit gagné.

01

Capacité

Est-ce que les machines vont continuer à progresser ?

- ? ON NE SAIT PAS

```
pour aller plus
loin
```

La durée d'une tâche qu'une IA mène seule, sans aide, double environ tous les 7 mois depuis 2019 (METR, mars 2025).

Une courbe qui double ne double jamais indéfiniment ; la vraie question est ce qui arrive au temps de doublement.

METR, « Measuring AI Ability to Complete Long Tasks » (mars 2025) ↗ · Kwa et al., arXiv:2503.14499 ↗ · vérifié le 2026-09-25

02

Demande

Est-ce que les gens vont s'en servir ?

☒ OUI, DÉJÀ

```
pour aller plus
.....
loin
```

Quand une ressource devient bon marché, on en consomme plus, pas moins : c'est le paradoxe de Jevons, observé sur le charbon en 1865.

Des machines à vapeur plus économes n'ont pas fait baisser la consommation de charbon britannique : elle a triplé d'ici 1900.

Jevons, « The Coal Question » (1865), Yale Energy History ↗ · Jevons paradox (Wikipedia) ↗ · vérifié le 2026-09-25

03

Ressources

Est-ce que
l'électricité et les
usines suivent ?

- ? C'EST LE FREIN

```
pour aller plus
*****
loin
```

Les data centers du monde ont consommé environ 415 TWh d'électricité en 2024 (IEA) ; l'Allemagne entière en a produit 489 TWh la même année : à peu près autant.

Un modèle se déploie en semaines, une usine de puces se construit en années : l'usine TSMC de l'Arizona annoncée en mai 2020, n'a produit en volume qu'à la fin 2024.

IEA, « Energy and AI »
(avril 2025) ↗ ·
Fraunhofer ISE,
production électrique
allemande 2024 ↗ ·
TSMC Arizona
(Wikipedia) ↗ · vérifié
le 2026-09-25

04

Capture

Qui va gagner
l'argent ?

- ? INCERTAIN

```
pour aller plus
*****
loin
```

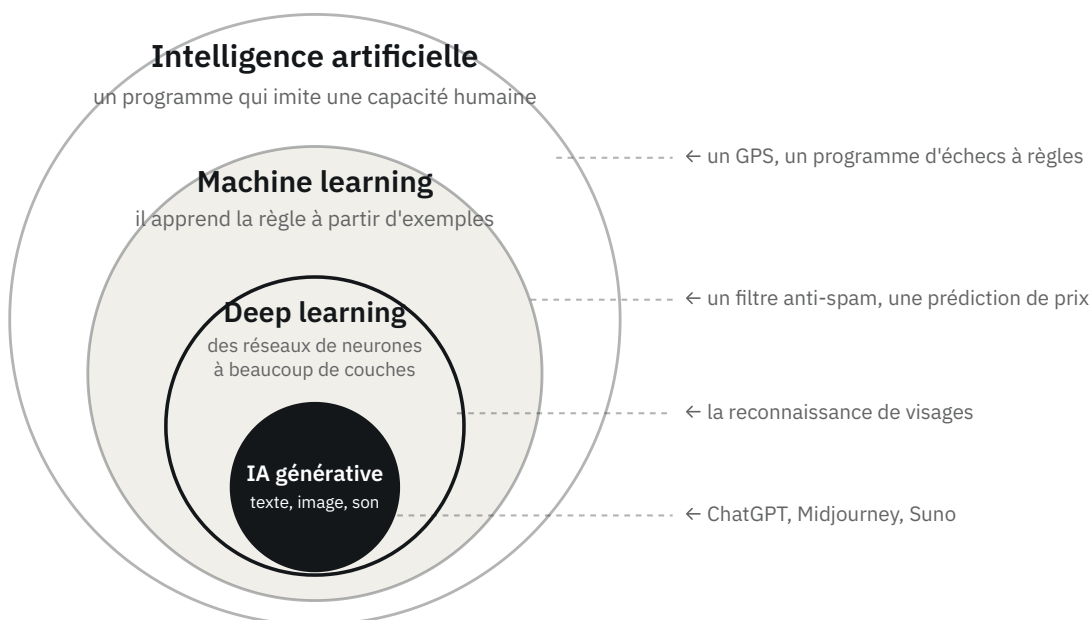
Le moteur électrique arrive dans les usines dans les années 1880 ; les usines américaines ne deviennent vraiment plus productives que dans les années 1920.

Le gain ne vient pas de l'outil, il vient de la réorganisation autour de l'outil : il a fallu reconstruire les ateliers autour de petits moteurs au lieu d'un arbre central.

Paul David, « The
Dynamo and the
Computer », *American
Economic Review*
80(2), 1990 ↗ · vérifié
le 2026-09-25

Les trois cercles

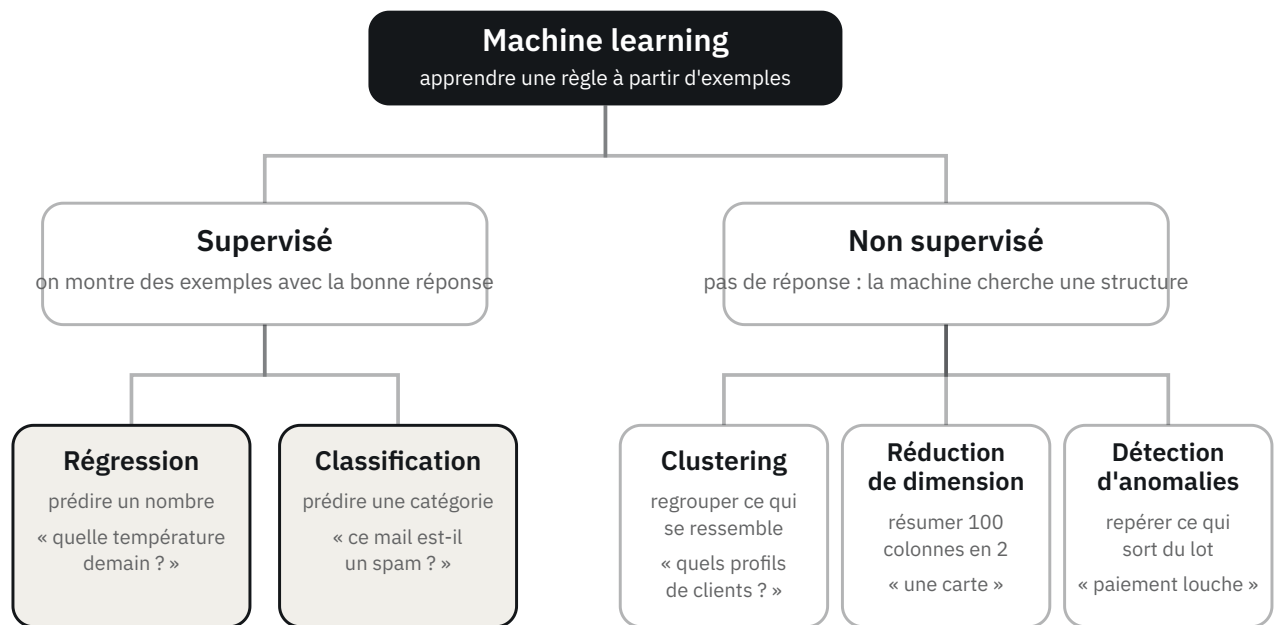
Trois mots que tout le monde mélange.



Chaque cercle est contenu dans le précédent : tout deep learning est du machine learning, tout machine learning est de l'IA, mais pas l'inverse.

La carte du machine learning

Quand quelqu'un dit « on va mettre du machine learning », il pense à l'une de ces cinq cases.



Les deux cases grisées sont celles du cours : c'est là que vivent Titanic, la météo et le prix des appartements.

- L'ANCÊTRE : LA RÉGRESSION LINÉAIRE, 1805

Le machine learning n'est pas de la magie : ce sont des mathématiques anciennes devenues calculables. Sa plus vieille méthode, la régression linéaire, consiste à faire passer la meilleure droite possible dans un nuage de points. Legendre l'a publiée en 1805 pour prédire la trajectoire des comètes, avec la méthode des moindres carrés, et Gauss a revendiqué l'avoir utilisée avant lui. Le mot « régression » vient de Francis Galton, en 1886, qui étudiait la taille des enfants par rapport à celle de leurs parents.

Deux siècles plus tard, c'est toujours le premier modèle qu'on entraîne (tu le feras à la séance 6), et un réseau de neurones n'est, au fond, qu'un empilement de régressions avec une petite courbure entre chaque couche. Ce qui a changé, ce n'est pas l'idée : c'est qu'on peut désormais la faire tourner sur des milliards de nombres.

Classification ou régression

C'est la distinction qu'on te demandera à chaque projet. Le réflexe : ne regarde pas le sujet de la question, regarde **la forme de la réponse**.

CLASSIFICATION OU RÉGRESSION ? • SUR PAPIER

Pour chaque question, dis si elle demande une catégorie (classification) ou un nombre (régression).

Classification : la réponse est une catégorie

Régression : la réponse est un nombre

1. Quelle température fera-t-il demain ?

2. Ce mail est-il un spam ?
3. Combien de vues fera cette vidéo ?
4. Cette photo montre-t-elle un chat ?
5. À quel prix se vendra cet appartement ?
6. Ce passager du Titanic a-t-il survécu ?
7. Quel genre musical est ce morceau ?
8. Dans combien de minutes arrivera le bus ?

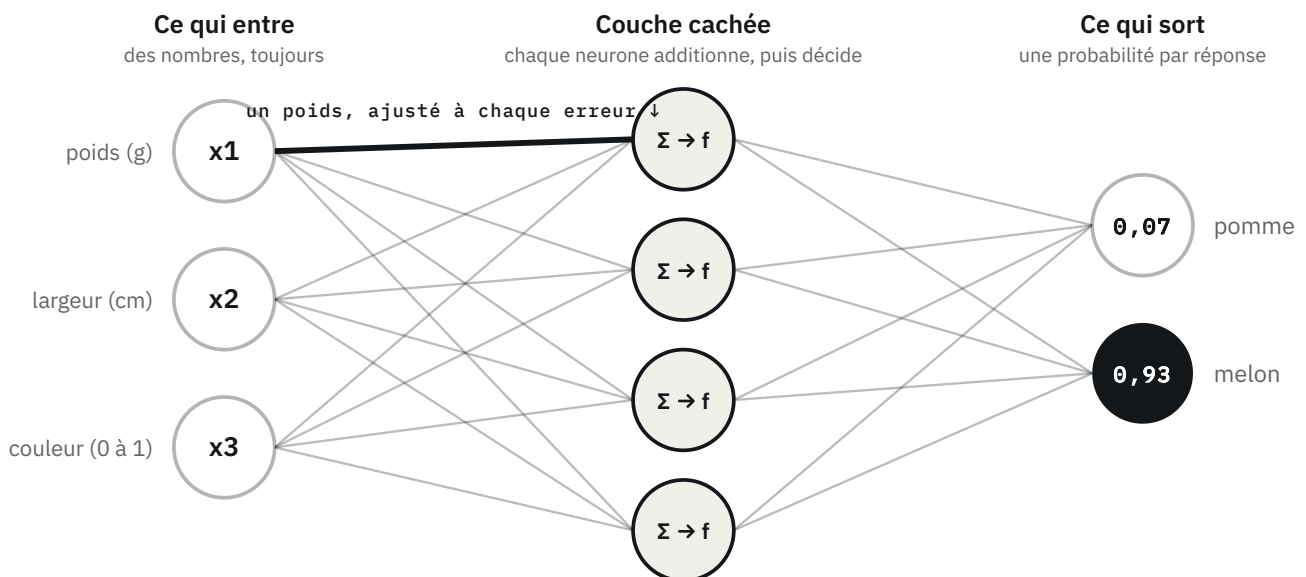
Les réponses sont dans le corrigé, en fin de leçon.

- POURQUOI ÇA COMPTE : ON NE LES MESURE PAS PAREIL

Les deux familles ne se mesurent pas de la même façon. Une classification se juge en pourcentage de bonnes cases ; une régression se juge à la distance entre la prédiction et la vraie valeur. Se tromper de famille, c'est mesurer son modèle avec la mauvaise règle.

Un réseau de neurones

Reprenons la pomme et le melon du notebook.



Apprendre = régler tous les poids pour que la sortie colle aux exemples.
Avec des millions de poids et des dizaines de couches, c'est du deep learning.

À toi : le Playground

Ouvre le TensorFlow Playground (il faut le réseau). Choisis le jeu de données en spirale, garde une couche cachée à deux neurones, lance : le réseau n'y arrive pas. Ajoute des couches et des neurones avec les boutons +, relance, et regarde la frontière épouser la spirale. C'est ça, *deep* : la profondeur apprend des formes compliquées.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Sur la frise, pourquoi 2012 est-elle la vraie rupture, plutôt que 1957 ou 2022 ?
 - A. C'est l'année où l'idée du neurone artificiel est trouvée
 - B. Un réseau de neurones gagne enfin, grâce aux cartes graphiques et à une grande base d'images
 - C. C'est la sortie de ChatGPT
 - D. Les chercheurs inventent le transformer
2. Pourquoi une carte graphique entraîne-t-elle un réseau de neurones plus vite qu'un processeur ?
 - A. Ses cœurs sont plus rapides un par un
 - B. Elle a plus de mémoire
 - C. Elle a des milliers de petits cœurs qui font la même opération en même temps
 - D. Elle est branchée directement à Internet
3. Un GPS qui calcule l'itinéraire le plus rapide, où le places-tu dans les trois cercles ?
 - A. Dans le deep learning : il utilise un réseau de neurones
 - B. Dans le machine learning : il apprend des trajets
 - C. Dans l'intelligence artificielle seulement : des règles écrites à la main, rien d'appris
 - D. Nulle part : ce n'est pas de l'IA
4. Dans un réseau de neurones, qu'est-ce qui change pendant l'apprentissage ?
 - A. Le nombre de couches
 - B. Les poids des liens entre neurones, un peu à chaque erreur
 - C. Les données d'entrée
 - D. La question posée au réseau

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Les idées datent de 1957, la rupture de 2012, l'accélération de 2017, avec deux hivers entre-temps.

2. Trois causes au boom, aucune ne suffit seule, du calcul, des données, de meilleurs algorithmes.
3. Un GPU gagne parce qu'il fait des milliers de fois la même multiplication en même temps.
4. Tout deep learning est du machine learning, tout machine learning est de l'IA, pas l'inverse.
5. Apprendre, pour un réseau de neurones, c'est régler ses poids un peu à chaque erreur.

Corrigé

Classification ou régression ?

1. **Régression** · La réponse est un nombre sur une échelle continue : 14,5 °C, 22 °C... Entre deux valeurs, il y en a toujours une autre.
2. **Classification** · La réponse est l'une de deux cases : spam ou pas spam. Pas de « à moitié spam ».
3. **Régression** · On prédit une quantité : 300, 12 000, 2 millions. La bonne réponse peut être « pas loin ».
4. **Classification** · Oui ou non : deux catégories. Avec chat, chien, lapin, on aurait trois catégories, c'est toujours une classification.
5. **Régression** · Un montant en euros : un nombre, sur une échelle continue.
6. **Classification** · Survécu ou non : c'est exactement la question du projet Kaggle des séances 7 et 8.
7. **Classification** · Rock, jazz, rap... : on choisit une étiquette dans une liste, même si la liste est longue.
8. **Régression** · Une durée : 3 minutes, 7 minutes et demie. Un nombre, pas une case.

Mini-quiz

1. **B. Un réseau de neurones gagne enfin, grâce aux cartes graphiques et à une grande base d'images** · L'idée date de 1957, ChatGPT de 2022 et le transformer de 2017. En 2012, AlexNet gagne le concours ImageNet, entraîné sur deux cartes de jeu vidéo : les moyens rattrapent enfin l'idée.
2. **C. Elle a des milliers de petits cœurs qui font la même opération en même temps** · Ses cœurs sont plus lents et plus bêtes pris un par un. Mais un réseau de neurones fait des millions de fois la même multiplication, et ça, des milliers de cœurs identiques le font d'un coup.
3. **C. Dans l'intelligence artificielle seulement : des règles écrites à la main, rien d'appris** · Il imite une capacité humaine, trouver un chemin, donc c'est de l'IA. Mais l'algorithme est écrit à la main, sans exemples : ni machine learning, ni deep learning. Le grand cercle, pas les petits.
4. **B. Les poids des liens entre neurones, un peu à chaque erreur** · L'architecture et les données sont fixées avant. Apprendre, c'est régler les poids : à chaque exemple raté, on les décale un peu, des millions de fois, jusqu'à ce que la sortie colle aux exemples.

Python pour la data

Rappels Python, puis un tableau de 800 Pokémon dans pandas : filtrer, trier, compter, regrouper, et trois graphiques qui répondent chacun à une question.

À LA FIN DE LA SÉANCE

Tu sauras lire un DataFrame avec head, shape et describe, et dire ce qu'est une ligne, une colonne, un index.

Tu sauras filtrer, trier, compter et regrouper un tableau en une ligne de pandas chacune.

Tu sauras choisir entre barres, nuage de points et histogramme selon la question posée.

Un CSV, c'est du texte

Le notebook charge 800 Pokémon depuis un fichier CSV. Ouvre-le dans un éditeur de texte avant pandas : ce n'est que du texte, avec des virgules.

pokemon.csv, ouvert dans un éditeur de texte

du texte, des virgules, une ligne par Pokémon

```
#,Name,Type 1,Type 2,Total,HP,Attack,...
1,Bulbasaur,Grass,Poison,318,45,49,...
2,Ivysaur,Grass,Poison,405,60,62,...
3,Venusaur,Grass,Poison,525,80,82,...
```

→ `pd.read_csv(url)` lit ce texte et fabrique le tableau

df, le DataFrame

les mêmes valeurs, rangées et numérotées

index		en-tête							
	idx#	Name	Type 1	Type 2	Total	HP	Attack		
	0	1	Bulbasaur	Grass	Poison	318	45	49	
	1	2	Ivysaur	Grass	Poison	405	60	62	← ligne
	2	3	Venusaur	Grass	Poison	525	80	82	
					colonne ↑				

colonne ↑

800 lignes × 13 colonnes : `df.shape` donne (800, 13)
une colonne seule, `df["Speed"]`, est une Series

Le CSV est du texte avec des virgules ; pandas en fait un tableau qu'on interroge.

Devant n'importe quel tableau, trois réflexes, dans cet ordre :

```
df.head()      # les 5 premières lignes
df.shape       # (800, 13) : 800 lignes, 13 colonnes
df.describe()  # min, max, moyenne de chaque colonne de nombres : vitesse max 180
```

- LES RAPPELS PYTHON EN UNE CARTE

Tout ce dont on a besoin en data tient en quatre briques.

```

vitesses = [45, 65, 43, 90, 110] # liste : plusieurs valeurs
dans l'ordre
pikachu = {"nom": "Pikachu", "type": "Electric", "vitesse": 90} # dictionnaire :
clé → valeur, une fiche
for v in vitesses: print(v) # boucle : on passe sur
chaque valeur
def moyenne(liste): return sum(liste) / len(liste) # fonction : une recette
réutilisable

```

Une liste de dictionnaires, c’est déjà presque un tableau : pandas fait le reste.

Cinq verbes

Un verbe, une cellule exécutée, une question posée au groupe. Le tableau sert de fiche à retrouver ensuite.

VERBE	LA QUESTION QU’IL POSE	LA LIGNE PANDAS	EXEMPLE POKÉMON
Filtrer	Lesquels respectent une condition ?	<code>df[df["Type 1"] == "Fire"]</code>	52 Pokémon de type Feu
Combiner	Deux conditions à la fois ?	<code>df[(df["Speed"] > 100) & (df["Attack"] > 100)]</code>	38 rapides et forts
Trier	Qui est premier ?	<code>df.sort_values("Speed", ascending=False)</code>	DeoxysSpeed Forme, 180 de vitesse
Compter	Combien de chaque ?	<code>df["Type 1"].value_counts()</code>	Water, 112 Pokémon
Regrouper	Une moyenne par catégorie ?	<code>df.groupby("Type 1")["Total"].mean()</code>	Dragon, 550,5 en moyenne

Regrouper

La question du jour : quel type est le plus puissant ?

1. les lignes, mélangées

df

Charmander	Fire	309
Charmeleon	Fire	405
Caterpie	Bug	195
Metapod	Bug	205
Dratini	Dragon	300
Dragonair	Dragon	420

2. un tas par type

.groupby("Type 1")

Dragon

Dratini	300
Dragonair	420

Fire

Charmander	309
Charmeleon	405

Bug

Caterpie	195
Metapod	205

3. une moyenne par tas

["Total"].mean()

Dragon 550,5
1er sur 18 types · 32 Pokémon

Fire 458,1
5e sur 18 types · 52 Pokémon

Bug 378,9
18e sur 18 types · 69 Pokémon

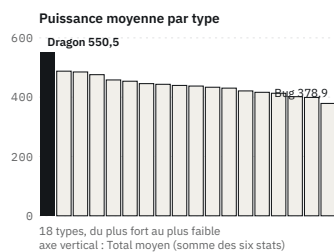
Une ligne de code, une réponse : le type le plus puissant est Dragon.

Regrouper, c'est faire des tas puis calculer une chose par tas.

Un graphique répond à une question

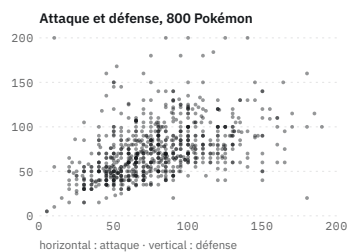
Pas de graphique sans titre ni axes : sans eux, personne ne sait à quelle question il répond.

BARRES · COMPARER DES CATÉGORIES



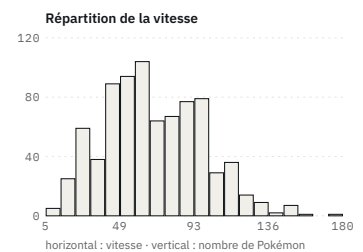
Quel type est le plus puissant ?
Dragon, loin devant ; Insecte ferme la marche.

NUAGE DE POINTS · UN LIEN ENTRE DEUX NOMBRES ?



Les Pokémon forts en attaque sont-ils solides en défense ?
Un peu : le nuage monte, sans s'aligner (corrélation 0,44).

HISTOGRAMME · COMMENT UNE VALEUR SE RÉPARTIT



Comment la vitesse se répartit-elle ? Une bosse entre 40 et 110, et quelques fusées au-delà de 150.

QUEL GRAPHIQUE ? · SUR PAPIER

Pour chaque question, choisis le graphique qui y répond.

Barres : comparer des catégories

Nuage de points : chercher un lien entre deux nombres

Histogramme : voir comment une valeur se répartit

1. Quel type de Pokémon est le plus fréquent ?

2. Les Pokémon forts en attaque sont-ils aussi solides en défense ?

3. Comment se répartissent les points de vie ?

4. Quelle génération a les Pokémon les plus rapides, en moyenne ?
5. Un Pokémon rapide est-il fragile ?
6. Les vitesses sont-elles serrées autour d'une valeur, ou très étalées ?

Les réponses sont dans le corrigé, en fin de leçon.

À toi : 3 questions, 3 graphiques

Ton premier projet de portfolio, en 25 minutes. Choisis trois questions qui t'intéressent vraiment sur les Pokémon ; pour chacune, une cellule de calcul, un graphique qui y répond, et la réponse en une phrase sous le graphique. Si tu bloques sur la syntaxe, demande le squelette de la ligne, pas la réponse.

Entre deux séances

Le carnet `02_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées. Il s'ouvre par un quiz de trois questions, celles qu'on se pose à voix haute au début de la séance suivante.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Le notebook affiche (800, 13) quand tu tapes `df.shape`. Ça veut dire quoi ?
 - A. 800 colonnes et 13 lignes
 - B. 800 lignes et 13 colonnes
 - C. 800 Pokémon dont 13 légendaires
 - D. Une erreur : il manque une colonne
2. Tu veux savoir quel type de Pokémon a la meilleure vitesse moyenne. Quelle ligne ?
 - A. `df.sort_values("Speed")`
 - B. `df["Type 1"].value_counts()`
 - C. `df.groupby("Type 1")["Speed"].mean()`
 - D. `df[df["Speed"] > 100]`
3. Quelle ligne garde les Pokémon à la fois rapides (`Speed > 100`) et forts (`Attack > 100`) ?
 - A. `df[df["Speed"] > 100 and df["Attack"] > 100]`
 - B. `df[(df["Speed"] > 100) & (df["Attack"] > 100)]`
 - C. `df[df["Speed"] > 100 & df["Attack"] > 100]`
 - D. `df.sort_values(["Speed", "Attack"])`

4. Un graphique en barres, sans titre ni nom d'axe, montre trois barres de hauteurs différentes. Que peut-on en conclure ?
- A. Que la première catégorie est la meilleure
 - B. Que les données sont fausses
 - C. Rien : on ne sait ni ce qui est compté, ni dans quelle unité
 - D. Qu'il faut plus de barres

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Un CSV est du texte avec des virgules ; pandas en fait un DataFrame, un tableau qu'on interroge.
2. Trois réflexes devant un tableau inconnu, `head()`, `shape`, `describe()`.
3. Cinq verbes, filtrer, combiner, trier, compter, regrouper, une ligne chacun.
4. `groupby` fait des tas par catégorie, puis calcule une chose par tas.
5. Un graphique répond à une question, et il a toujours un titre et des axes.

Corrigé

Quel graphique ?

1. **Barres** · On compare des catégories : une barre par type, la plus haute gagne. Ici Water, 112 Pokémon.
2. **Nuage de points** · Deux nombres par Pokémon, attaque et défense : un point chacun, et on regarde si le nuage monte. Il monte un peu, corrélation 0,44.
3. **Histogramme** · Une seule valeur, les HP, et on veut voir la forme : où sont la plupart des Pokémon, où sont les rares. La moitié a 65 HP ou moins, un seul monte à 255.
4. **Barres** · Une moyenne par génération, six catégories à comparer : des barres. La première génération est en tête, 72,6 de vitesse moyenne.
5. **Nuage de points** · Vitesse contre défense, deux nombres : un nuage de points. S'il descendait nettement, les rapides seraient fragiles ; il est très dispersé.
6. **Histogramme** · Une seule valeur, la vitesse, et on veut voir sa forme. La moitié centrale des Pokémon tient entre 45 et 90, mais la queue court jusqu'à 180 : étalée vers la droite.

Mini-quiz

1. **B. 800 lignes et 13 colonnes** · shape donne toujours (lignes, colonnes) dans cet ordre. 800 Pokémon, 13 mesures par Pokémon. Les légendaires, eux, sont 65 : ça se compte avec un filtre, pas avec shape.
2. **C. `df.groupby("Type 1")["Speed"].mean()`** · « Par type » appelle groupby : un tas par type, puis la moyenne de la vitesse dans chaque tas. Trier donne les Pokémon un par un, value_counts compte les types, le filtre garde les rapides sans les regrouper.
3. **B. `df[(df["Speed"] > 100) & (df["Attack"] > 100)]`** · pandas relie deux conditions avec &, chacune entre ses parenthèses. Sans elles, Python calcule d'abord `100 & df["Attack"]` et la ligne plante ; « and » ne sait pas comparer deux colonnes entières et plante aussi ; `sort_values` range, il ne filtre pas. La bonne ligne garde 38 Pokémon.
4. **C. Rien : on ne sait ni ce qui est compté, ni dans quelle unité** · Sans titre ni axes, on ne sait pas à quelle question le graphique répond, ni ce que mesure la hauteur. C'est la règle de la séance : pas de graphique sans titre ni axes.

SQL et Git

Où une appli range ses données, SQL pour interroger une base (SELECT, WHERE, GROUP BY, un premier JOIN), puis Git et GitHub pour mettre son premier projet en ligne.

À LA FIN DE LA SÉANCE

- Tu sauras dire où une appli range ses données et lire une requête SQL comme une phrase.
- Tu sauras filtrer, trier, compter, regrouper et relier deux tables, en SQL comme en pandas.
- Tu sauras ce que font add, commit, push et pull, et pousser un notebook sur GitHub depuis Colab.

Où vivent les données d'une appli

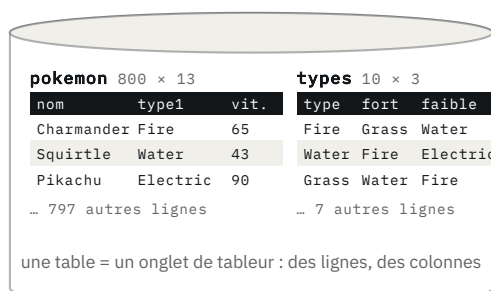
Derrière Instagram, Doctolib ou ton jeu, pas de fichier CSV : une base de données, faite de tables, et une langue pour l'interroger.

1. l'appli, ce que tu vois



une liste, un score, un profil...

2. la base de données, ce que tu ne vois pas



ici SQLite, une base complète qui tient dans un fichier ; ailleurs des serveurs entiers

3. SQL, la langue pour lui parler

```
SELECT nom FROM pokemon
WHERE type1 = 'Fire'
```

52 lignes, les Pokémon Feu

une question, un tableau en réponse

Une appli garde ses données dans des tables ; SQL est la langue pour leur poser des questions.

- SQL A CINQUANTE ANS, SQLITE EST DANS TA POCHE

SQL est né chez IBM en 1974, sous le nom SEQUEL, et tout le monde le parle encore : les sites, les banques, les trains. Il ne fait pas tourner l'ordinateur, il pose des questions à des tables ; c'est pour ça qu'il a survécu à tous les langages à la mode.

SQLite est une base de données complète qui tient dans un seul fichier, sans serveur. Chaque téléphone en contient : les contacts, les messages, l'historique du navigateur y sont rangés. En Python, le module `sqlite3` est fourni d'office : le notebook crée sa base en mémoire, sans rien installer.

Une requête se lit comme une phrase

la question, en SQL

```
SELECT nom, vitesse
FROM pokemon
WHERE type1 = 'Fire'
ORDER BY vitesse DESC
LIMIT 3
```

une seule phrase : « les 3 Pokémon Feu les plus rapides »

ce que dit chaque ligne

- ... quelles colonnes
- ... quelle table
- ... quelle condition sur les lignes
- ... dans quel ordre
- ... combien de lignes

la réponse, un tableau

nom	vitesse
Talonflame	126
Infernape	108
Pyroar	106

3 lignes gardées sur les 52 Pokémon Feu,
2 colonnes gardées sur 13

sans WHERE, le tableau ferait 800 lignes ;
sans LIMIT, 52.

Une requête se lit comme une phrase : quoi, d'où, à quelle condition, dans quel ordre, combien.

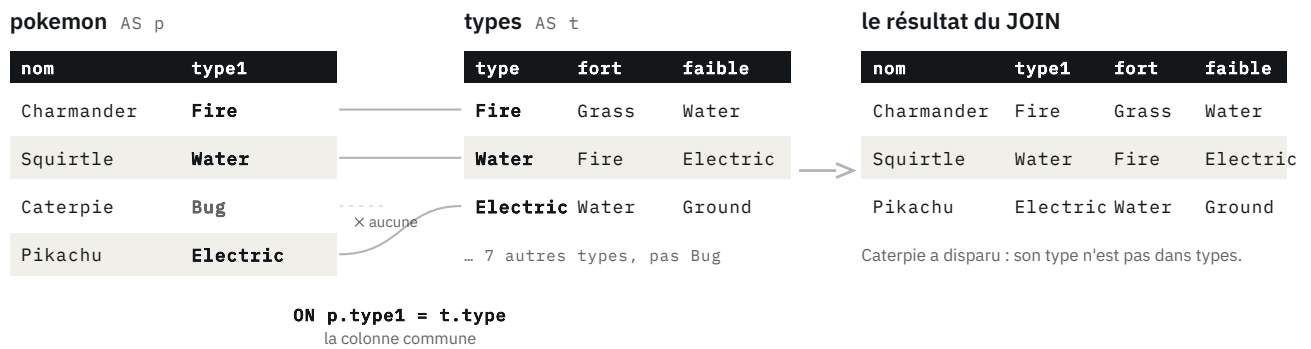
```
sql("SELECT nom, vitesse FROM pokemon WHERE type1 = 'Fire' ORDER BY vitesse DESC
LIMIT 3")
```

Les verbes de la séance 2 ont chacun leur traduction. Le tableau sert de fiche à retrouver ensuite.

LE VERBE	EN SQL	EN PANDAS	EXEMPLE
Filtrer	WHERE type1 = 'Fire'	df[df["type1"] == "Fire"]	52 Pokémon Feu
Trier	ORDER BY vitesse DESC LIMIT 5	df.sort_values("vitesse", ascending=False).head(5)	DeoxysSpeed Forme, 180
Compter	SELECT COUNT(*) ... WHERE legendaire = 1	len(df[df["legendaire"] == 1])	65 légendaires
Regrouper	SELECT type1, AVG(total) ... GROUP BY type1	df.groupby("type1") ["total"].mean()	Dragon, 550,5
Relier	JOIN types AS t ON p.type1 = t.type	df.merge(types, left_on="type1", right_on="type")	466 lignes recollées

Relier deux tables

Une vraie base éparpille l'information : les Pokémon d'un côté, les forces et faiblesses de chaque type de l'autre.



Sur les 800 Pokémon du notebook, **466** trouvent leur type dans la table types ; **334** restent dehors. `LEFT JOIN` les garde, avec des cases vides.

`JOIN` recolle deux tables par leur colonne commune ; ce qui n'a pas de correspondance disparaît.

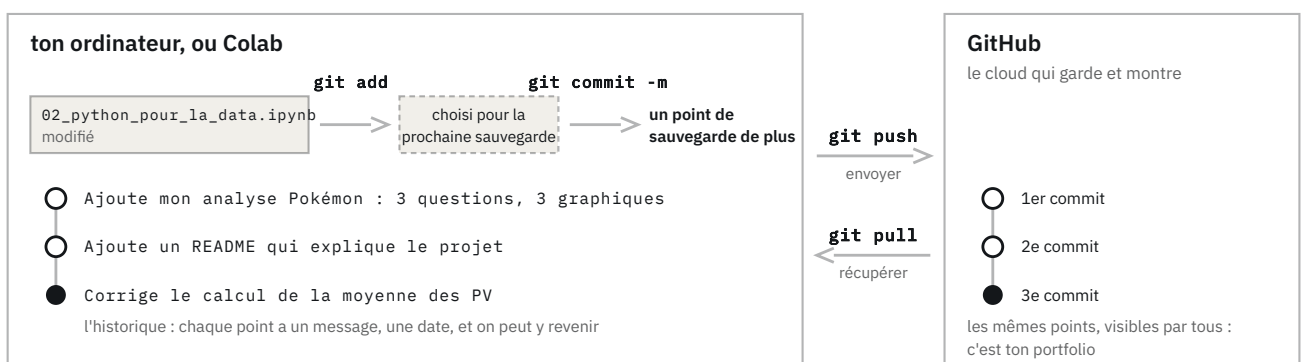
- LES OUBLIÉS DU JOIN

Huit types du dataset n'ont pas de ligne dans la table `types` : Bug, Dark, Dragon, Fairy, Ghost, Normal, Poison, Steel. Leurs 334 Pokémon disparaissent d'un `JOIN` simple, sans message d'erreur. C'est le piège classique : un résultat plus petit que prévu, et personne ne s'en aperçoit.

`LEFT JOIN` garde toutes les lignes de la table de gauche ; celles qui n'ont pas de correspondance ont des cases vides (`NULL`). Pour compter les oubliés : `WHERE t.type IS NULL` .

Git : des points de sauvegarde

Tu connais les sauvegardes d'un jeu vidéo : on y revient quand on rate. Git fait ça pour ton code ; GitHub garde les sauvegardes en ligne et les montre.



Pour chaque situation, choisis la commande Git qui convient.

git add : choisir les fichiers qui entreront dans la prochaine sauvegarde

git commit -m : créer le point de sauvegarde, avec un message qui dit pourquoi

git push : envoyer ses sauvegardes sur GitHub

git pull : récupérer ce qui a changé sur GitHub

1. Tu veux choisir quels fichiers modifiés entreront dans la prochaine sauvegarde.
2. Tu veux créer le point de sauvegarde, avec un message qui dit ce que tu as fait.
3. Tes sauvegardes sont sur ton ordinateur ; tu veux qu'elles apparaissent sur GitHub.
4. Tu ouvres ton projet sur un autre ordinateur et tu veux la dernière version qui est sur GitHub.
5. Tu as trois fichiers modifiés, mais tu ne veux en sauvegarder qu'un seul.
6. Dans Colab, « Enregistrer une copie dans GitHub » revient à trois commandes d'un clic. Laquelle n'en fait pas partie ?

Les réponses sont dans le corrigé, en fin de leçon.

À toi : ton projet en ligne

Depuis Colab, le menu Fichier, puis « Enregistrer une copie dans GitHub », revient à add, commit et push d'un clic.

Pousse ton notebook de la séance 2 dans ton dépôt `mon-portfolio-ia`, chemin `seance-02/02_python_pour_la_data.ipynb`, avec un message qui dit ce qu'il contient.

Écris le README : la cellule de la section 6 le génère depuis ta fiche, tu le colles sur GitHub. Modifie-le une deuxième fois : ton historique a deux versions.

Même question, deux langues (section 7) : trois défis à résoudre en SQL puis en pandas, compare dit si les deux réponses concordent. Le premier est résolu, les deux autres sont à toi.

Entre deux séances

Le carnet `03_exercices.ipynb` : 12 exercices, de 🌟 à 🌟🌟🌟, avec vérification automatique, indices et solutions repliées, dont deux sur Git et deux où SQL et pandas doivent tomber d'accord. Il s'ouvre par un quiz de trois questions, celles qu'on se pose à voix haute au début de la séance suivante.

Mini-quiz

MINI-QUIZ • SUR PAPIER

Une seule bonne réponse par question.

1. Tu veux les 3 Pokémon Feu les plus rapides. Quelle partie de la requête garde seulement les Pokémon Feu ?
 - A. SELECT nom, vitesse
 - B. WHERE type1 = 'Fire'
 - C. ORDER BY vitesse DESC
 - D. LIMIT 3
2. En pandas tu écrirais `df.groupby("type1")["total"].mean()`. En SQL ?
 - A. SELECT type1, total FROM pokemon ORDER BY total
 - B. SELECT AVG(total) FROM pokemon WHERE type1
 - C. SELECT type1, AVG(total) FROM pokemon GROUP BY type1
 - D. SELECT COUNT(*) FROM pokemon GROUP BY total
3. Un JOIN entre pokemon (800 lignes) et types (10 types) renvoie 466 lignes. Où sont passés les 334 autres Pokémon ?
 - A. SQL les a perdus par erreur
 - B. Leur type n'est pas dans la table types : un JOIN simple les laisse tomber
 - C. Ce sont les Pokémon légendaires
 - D. Ils étaient en double
4. Dans quel ordre tapes-tu ces trois commandes pour envoyer ton notebook sur GitHub ?
 - A. commit, add, push
 - B. add, commit, push
 - C. push, add, commit
 - D. add, push, commit

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Une appli garde ses données dans une base faite de tables ; SQL est la langue pour les interroger.
2. Une requête se lit comme une phrase, SELECT quoi, FROM où, WHERE à quelle condition, ORDER BY dans quel ordre, LIMIT combien.
3. SQL et pandas font la même chose, WHERE et filtre, ORDER BY et sort_values, GROUP BY et groupby, JOIN et merge.
4. Un JOIN recolle deux tables par une colonne commune, et laisse tomber ce qui n'a pas de correspondance.
5. Git garde des points de sauvegarde, GitHub les met en ligne ; add choisit, commit sauvegarde avec un message qui dit pourquoi, push envoie, pull récupère.

Corrigé

Quelle commande ?

1. **git add** · add ne sauvegarde rien encore : il met de côté ce qui fera partie du prochain point de sauvegarde. Le commit vient après.
2. **git commit -m** · Le commit enregistre une version. Son message est lu par celui qui relira l'historique : « Corrige le calcul de la moyenne des PV » dit tout, « modif » ne dit rien.
3. **git push** · Un commit reste sur la machine où il a été fait. push copie tes commits sur GitHub ; sans lui, ton dépôt en ligne ne bouge pas.
4. **git pull** · pull rapatrie ce qui a changé sur GitHub, dans l'autre sens que push. Dans Colab, ouvrir un notebook depuis l'onglet GitHub revient à un pull.
5. **git add** · C'est exactement le rôle de add : choisir. Tu ajoutes le seul fichier voulu, puis le commit ne contient que lui.
6. **git pull** · Le clic revient à add, commit et push : Colab prend le notebook, écrit le point de sauvegarde avec ton message directement dans ton dépôt GitHub. pull, c'est l'inverse : récupérer.

Mini-quiz

1. **B. WHERE type1 = 'Fire'** · SELECT choisit les colonnes, WHERE les lignes, ORDER BY l'ordre, LIMIT le nombre. Sans WHERE, la requête renverrait les trois plus rapides de tous les types, DeoxysSpeed Forme en tête avec 180.
2. **C. SELECT type1, AVG(total) FROM pokemon GROUP BY type1** · GROUP BY fait les tas, un par type ; AVG calcule la moyenne dans chaque tas. Les deux langues donnent le même classement, Dragon en tête avec 550,5.
3. **B. Leur type n'est pas dans la table types : un JOIN simple les laisse tomber** · Bug, Normal, Poison et cinq autres types n'ont pas de ligne dans types, donc pas de correspondance. Un JOIN simple ne garde que ce qui se recolle ; LEFT JOIN garderait les 800, avec des cases vides.
4. **B. add, commit, push** · add choisit les fichiers, commit crée le point de sauvegarde avec son message, push envoie les commits sur GitHub. Un push avant le commit n'a rien de nouveau à envoyer.

Collecter et nettoyer

Une journée de data scientist : réparer un tableau Pokémon abîmé exprès en tenant le journal de chaque correction, puis aller chercher des données soi-même avec une API.

À LA FIN DE LA SÉANCE

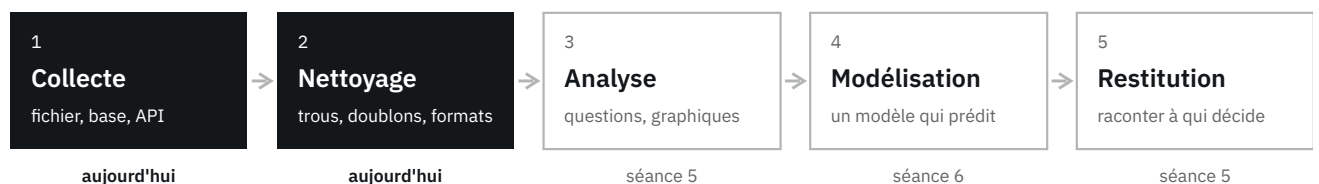
Tu sauras reconnaître les cinq dégâts d'un vrai tableau, doublons, fautes de frappe, nombres en texte, cases vides, dates mélangées.

Tu sauras réparer chacun en une ou deux lignes de pandas, et décider s'il faut supprimer, remplir ou garder une case vide.

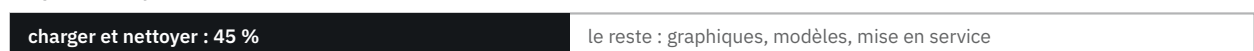
Tu sauras appeler une API, lire sa réponse JSON et en faire un tableau.

Une journée de data scientist

On imagine des graphiques et des modèles. La réalité commence plus tôt, avec des données sales.



Où passe une journée de travail



Enquête Anaconda 2020, près de 2 400 répondants : 19 % pour charger, plus d'un quart pour nettoyer.

Cinq étapes, et près de la moitié du temps passe dans les deux premières.

– D'OÙ VIENT LE CHIFFRE DE 45 %

En 2020, l'éditeur Anaconda a interrogé près de 2 400 personnes qui font de la data dans plus de cent pays. En moyenne, elles passent 19 % de leur temps à charger les données et plus d'un quart à les nettoyer, avant de pouvoir s'en servir ; les graphiques prennent environ 21 %, chaque étape du modèle 11 à 12 %.

Le rapport Anaconda 2020 ↗ • un résumé de l'enquête ↗

Cinq dégâts

Le notebook prend les 800 Pokémon propres de la séance 2 et les abîme exprès, comme la vraie vie le ferait. Tout le monde obtient le même tableau abîmé.

df, après abimer(seed=42)

six lignes réelles sur 810

Name	Type 1	HP	date_capture
Palpitoad	water 2	"75_" 3	05/03/2024
Charizard	Fier 2	"78_"	10/05/2024
Charmeleon	fire	NaN 4	2024-10-17
1 Growlithe	Fire	"55_"	28 sept 2024 5
1 Growlithe	Fire	"55_"	28 sept 2024
Chandelure	Ghost	"60_"	14 mars 2024 5

_ marque un espace invisible à la fin du texte.

- 1 doublons
810 lignes au lieu de 800
10 lignes recopiées, comme Growlithe
- 2 fautes de frappe
56 valeurs de Type 1 au lieu de 18
water, Fire_, Fier : trois « Feu » pour l'ordinateur
- 3 nombres en texte
HP est une colonne de texte
« 75_ » avec un espace : pas de moyenne possible
- 4 cases vides
40 Type 1, 41 HP
NaN, Not a Number : la case a été effacée
- 5 dates en trois formats
255 + 271 + 274
2024-10-17, 05/03/2024, 28 sept 2024

Cinq dégâts, ceux que la vraie vie fait subir à tout tableau.

DÉGÂT	COMMENT LE VOIR	COMMENT LE RÉPARER	SUR LE POKÉMON ABÎMÉ
Doublons	df.duplicated().sum() ()	df.drop_duplicates()	10 lignes en trop
Fautes de frappe	df["Type 1"].unique()	.str.strip().str.capitalize() , puis un dictionnaire des fautes	56 valeurs, puis 30, puis 18
Nombres en texte	df["HP"].dtype affiche object	.str.strip().astype(float)	« 75 » devient 75
Cases vides	df.isna().sum()	fillna , dropna , ou rien	40 Type 1, 40 HP
Dates mélangées	trois formats à l'œil	pd.to_datetime(..., format=..., errors="coerce")	0 date illisible à la fin

Supprimer, remplir ou garder

Une case vide n'est pas toujours un bug : il faut d'abord comprendre pourquoi elle manque.

SUPPRIMER, REMPLIR OU GARDER ? • SUR PAPIER

Pour chaque case vide, choisis ce qu'on en fait.

Supprimer : la case vide rend la ligne inutile (dropna)

Remplir : on met quelque chose à la place de la valeur perdue : la médiane pour un nombre, l'étiquette « Inconnu » pour une catégorie, jamais une valeur devinée (fillna)

Garder : la case vide porte elle-même une information, comme Type 2 pour un Pokémon à un seul type ; on la laisse telle quelle

1. 386 Pokémon sur 800 n'ont rien dans la colonne Type 2.
2. 40 Pokémon ont perdu leurs points de vie (HP).
3. 40 Pokémon ont perdu leur Type 1.
4. Dans Tips, 12 additions n'ont plus de montant (total_bill).
5. Dans Tips, 12 additions ont perdu leur jour de la semaine.
6. Une ligne n'a plus que son nom : toutes les autres cases sont vides.

Les réponses sont dans le corrigé, en fin de leçon.

Le journal des corrections

Si on te demande dans six mois pourquoi il n'y a plus que 800 lignes, tu dois pouvoir répondre. Le notebook tient une simple liste, remplie au fil des réparations.

```
journal = []
def noter(message):
    journal.append(message)
```

1. le journal, une ligne par réparation

ce que le notebook affiche à la fin, dans l'ordre

1. Départ : 810 lignes, 471 cases vides
2. Doublons : 10 lignes identiques supprimées, il reste 800 lignes
3. Type 1 : espaces et majuscules normalisés, 12 fautes corrigées → 18 types valides
4. HP : texte converti en nombre (strip + astype)
5. HP : 40 cases vides remplies par la médiane (65)
6. Type 1 : cases vides remplacées par 'Inconnu'
7. date_capture : 3 formats convertis en vraies dates (to_datetime)

2. la preuve

le tableau réparé face au tableau propre

	propre	réparé
lignes	800	800
types de Type 1	18	19 dont « Inconnu »
HP moyen	69,3	69,1

L'écart de HP vient des 40 cases remplies par 65 : le journal dit pourquoi le chiffre a bougé.

Chaque réparation s'écrit ; le tableau réparé se compare au tableau d'origine.

Une API : aller chercher ses données

Une API est une porte par laquelle un programme demande des données à un site : au restaurant, tu ne vas pas en cuisine, tu passes commande.

1. la commande : une URL

```
pokeapi.co/api/v2/pokemon/pikachu
```

`requests.get(url)` passe la commande,
`.json()` ouvre le plat

2. le plat : du JSON, des dictionnaires dans des listes

```
{
  "name": "pikachu",
  "height": 4,
  "weight": 60,
  "types": [ { "type": { "name": "electric" } } ],
  "stats": [ { "stat": { "name": "hp" }, "base_stat": 35 },
              { "stat": { "name": "attack" }, "base_stat": 55 }, ... ],
  ... 16 autres clés
}
```

le type est rangé trois niveaux plus bas :

`data["types"][0]["type"]["name"] → "electric"`

3. une ligne de DataFrame

un appel par Pokémon, une ligne chacun

nom	type	taille	poids	hp	attack	defense	speed
pikachu	electric	4	60	35	55	40	90

Une API répond en JSON ; on y va chercher ce qu'il faut pour remplir une ligne de tableau.

- ET SI LE RÉSEAU TOMBE ?

Un appel réseau peut échouer : pas de connexion, site en panne, Pokémon mal orthographié. La fonction `demandeur` du notebook enveloppe l'appel dans un `try/except` et renvoie une réponse de secours écrite à l'avance, pour que la suite du notebook tourne quand même. La deuxième API du notebook, Open-Meteo, donne la météo d'une latitude et d'une longitude, gratuitement et sans clé.

Dans la PokéAPI, `height` est en décimètres et `weight` en hectogrammes : Pikachu mesure 0,4 m et pèse 6 kg. Lire la documentation d'une API, c'est aussi apprendre ses unités.

À toi : nettoie Tips

Tips, ce sont 244 additions d'un restaurant américain. Abîmées par la même fonction, elles deviennent 254 lignes, avec 14 façons d'écrire les 4 jours de la semaine.

Répare dans l'ordre : doublons, catégories, nombres, cases vides, dates, une ligne de journal par correction.

Justifie tes cases vides dans le journal : une addition sans montant, un jour effacé, que fais-tu de chacun ?

Prouve ton travail en comparant au tableau propre, puis sauvegarde `tips_propre.csv` : c'est ton livrable.

Entre deux séances

Le carnet `04_exercices.ipynb` : 12 exercices, de 🌟 à 🌟🌟🌟, avec vérification automatique, indices et solutions repliées. Il reprend la réparation du Pokémon étape par étape, une API avec son plan B, et finit par Tips de A à Z.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Dans le tableau abîmé, « fire », « Fire » suivi d'un espace, et « Fire » comptent comme trois types différents. Quelle ligne en fait un seul ?
 - A. `df["Type 1"].str.strip().str.capitalize()`
 - B. `df["Type 1"].dropna()`
 - C. `df.drop_duplicates()`
 - D. `df["Type 1"].astype(float)`
2. Pour convertir la colonne HP, le notebook écrit `astype(float)` et pas `astype(int)`. Pourquoi ?
 - A. Les points de vie ont des virgules
 - B. Il reste des cases vides, et NaN ne rentre pas dans une colonne d'entiers
 - C. float est plus rapide
 - D. int ne sait pas lire le texte
3. Après réparation, le HP moyen vaut 69,1 contre 69,3 dans le tableau propre. Qu'est-ce que ça veut dire ?
 - A. Le nettoyage a raté
 - B. Il manque des Pokémon
 - C. Les 40 cases remplies par 65 ne valent pas les vraies valeurs effacées : la moyenne bouge un peu
 - D. pandas arrondit mal
4. La PokéAPI renvoie data, un JSON. Quelle ligne donne le type de Pikachu ?
 - A. `data["type"]`
 - B. `data["types"]["name"]`
 - C. `data["types"][0]["type"]["name"]`
 - D. `data.type`

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Un vrai tableau est sale ; le préparer prend près de la moitié du temps du métier.
2. Le bilan de santé d'abord, info, isna, duplicated, unique, avant de toucher à quoi que ce soit.
3. Chaque dégât a son outil, drop_duplicates, strip et capitalize, astype, fillna ou dropna, to_datetime.
4. Une case vide se supprime, se remplit ou se garde, et le choix se justifie dans le journal.
5. Une API, c'est une URL, une réponse JSON, puis un tableau ; toujours avec un plan B si le réseau tombe.

Corrigé

Supprimer, remplir ou garder ?

1. **Garder** · Ce n'est pas un trou : ces Pokémon n'ont qu'un seul type. Le vide porte une information, le remplir l'effacerait.
2. **Remplir** · Le reste de la ligne est précieux. La médiane des HP, 65, est une valeur raisonnable qui ne dépend pas des rares géants, jusqu'à 255.
3. **Remplir** · Ici le vide ne dit rien en soi : le type a été effacé. La ligne reste utile, mais on ne devine pas son type : la case reçoit l'étiquette « Inconnu ». Écrire « Water » au hasard serait mentir.
4. **Supprimer** · Une addition sans montant ne sert à rien pour étudier les pourboires. 12 lignes sur 244, on les retire et on l'écrit dans le journal.
5. **Remplir** · Le jour a été effacé, le vide ne dit rien en soi. Le montant et le pourboire restent utiles : le jour reçoit l'étiquette « Inconnu », et chaque calcul par jour le montrera à part.
6. **Supprimer** · Il n'y a plus rien à analyser, et remplir toutes ces cases reviendrait à fabriquer un Pokémon. Une ligne vide va à la corbeille, avec sa ligne de journal.

Mini-quiz

1. **A. `df["Type 1"].str.strip().str.capitalize()`** · `strip` enlève l'espace invisible au bout, `capitalize` remet une majuscule devant et des minuscules derrière. Le tableau passe de 56 valeurs à 30 ; les lettres inversées comme « Fier » demandent ensuite un dictionnaire de corrections.
2. **B. Il reste des cases vides, et NaN ne rentre pas dans une colonne d'entiers** · NaN est lui-même un nombre à virgule : une colonne d'entiers ne peut pas le contenir. On passe par float, on remplit les vides, et seulement ensuite on repasse en int.
3. **C. Les 40 cases remplies par 65 ne valent pas les vraies valeurs effacées : la moyenne bouge un peu** · Remplir, c'est remplacer une valeur perdue par une valeur raisonnable, pas par la vraie. Un petit écart est normal ; ce qui compte, c'est que le journal dise d'où il vient.
4. **C. `data["types"][0]["type"]["name"]`** · `types` est une liste (un Pokémon peut en avoir deux), d'où le `[0]` ; chaque élément est un dictionnaire qui contient `type`, lui-même un dictionnaire qui contient `name`. On descend niveau par niveau jusqu'à « electric ».

Analyser et raconter

L'analyse exploratoire comme une enquête : repérer une tendance, une anomalie, une corrélation, déjouer trois pièges, puis construire un mini-dashboard et le présenter à un client en deux minutes.

À LA FIN DE LA SÉANCE

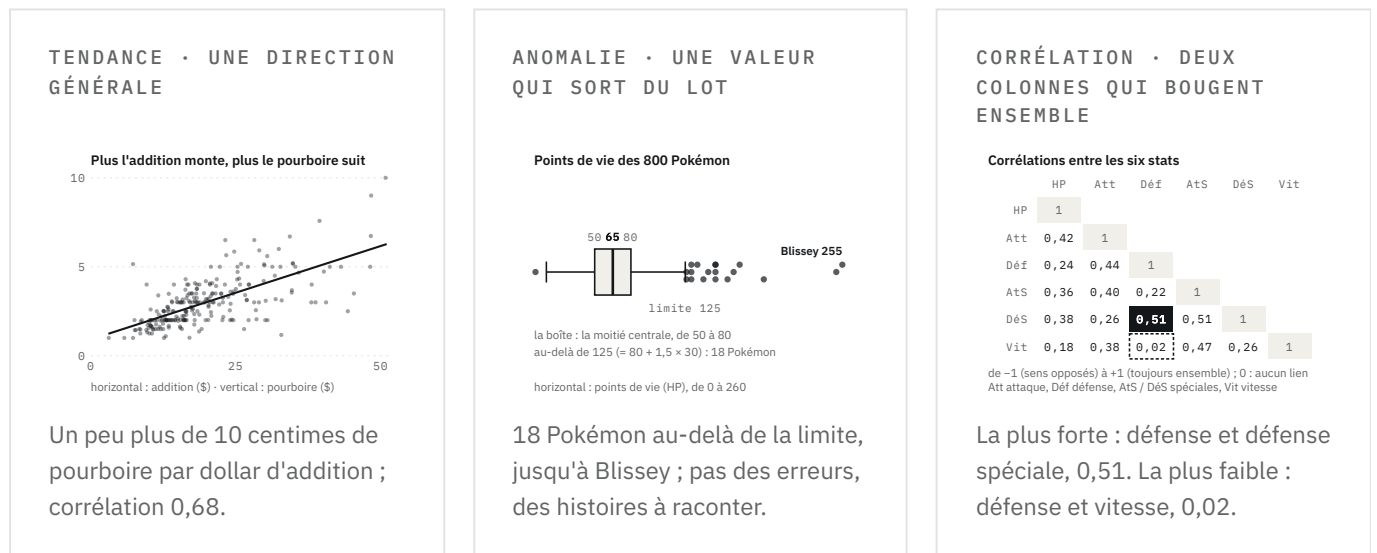
Tu sauras repérer une tendance, une anomalie et une corrélation, avec le graphique qui montre chacune.

Tu sauras reconnaître trois pièges, une corrélation lue comme une cause, un graphique trompeur, un échantillon biaisé.

Tu sauras assembler un tableau de bord Plotly et le présenter en trois temps, constat, preuve, recommandation.

Une enquête, trois indices

Un tableau de 800 lignes ne dit rien à personne. L'analyse exploratoire, c'est faire connaissance avec les données avant de conclure : on regarde tout, on note ce qui est bizarre.



- LA RÈGLE DES 1,5 ÉCARTS

On mesure l'écart entre le premier quart des valeurs et le troisième : pour les HP, de 50 à 80, soit 30. Tout ce qui dépasse le troisième quart de plus d'une fois et demie cet écart est suspect : au-delà de 80 + 45 = 125. Même chose par le bas : Shedinja, avec 1 HP, sort aussi de la boîte.

Une anomalie n'est pas forcément une erreur. Blissey a vraiment 255 HP : c'est souvent la chose la plus intéressante à raconter.

Trois pièges

Une corrélation, un graphique et un échantillon peuvent tous raconter une histoire fausse avec des chiffres justes.



QUEL PIÈGE ? · SUR PAPIER

Pour chaque affirmation, nomme le piège qui s'y cache.

Causalité : on lit une cause dans une corrélation, alors qu'une variable cachée peut expliquer les deux

Graphique trompeur : le dessin grossit ou écrase un écart : axe qui ne part pas de zéro, perspective, échelle trafiquée

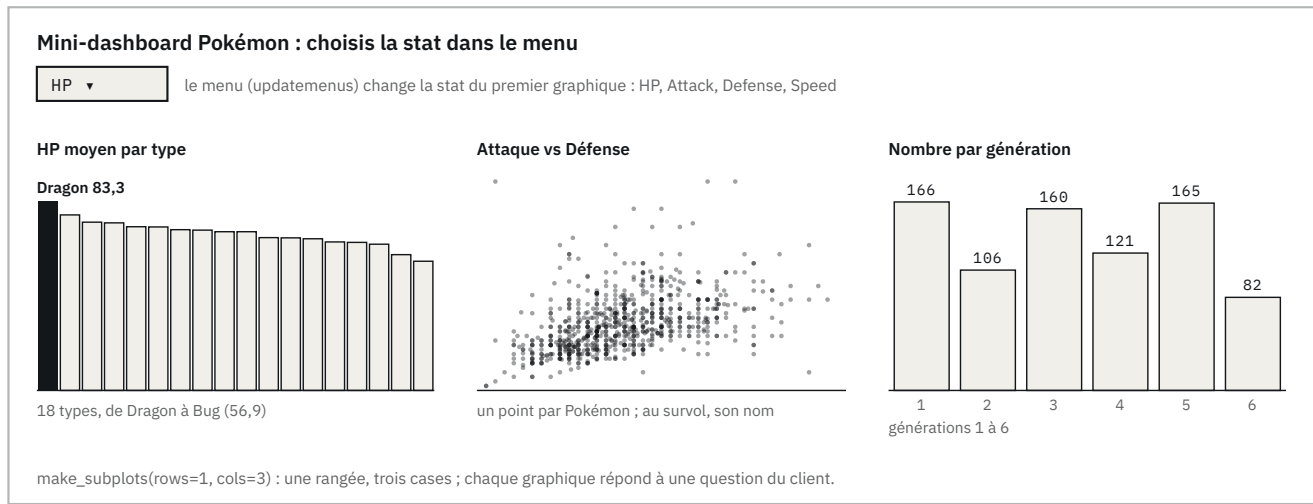
Biais : les données viennent d'un groupe qui ne ressemble pas à celui dont on parle

1. Les élèves qui ont un ordinateur portable ont de meilleures notes : donnons un portable à tout le monde !
2. Notre appli est passée de 4,1 à 4,3 étoiles : sur le graphique, la barre est deux fois plus haute.
3. 95 % des avis sur notre jeu sont positifs ; seuls les joueurs qui l'ont fini ont été invités à noter.
4. Les enfants qui dorment avec une veilleuse deviennent plus souvent myopes : éteignons les veilleuses.
5. Pour savoir si les élèves aiment la cantine, on interroge ceux qui y mangent, un jour de frites.
6. Sur un camembert en 3D penché, la part du premier plan paraît plus grosse qu'une part égale au fond.

Les réponses sont dans le corrigé, en fin de leçon.

Un tableau de bord

Plotly rend chaque graphique interactif : on survole un point, on zoome, on masque une catégorie en cliquant sur la légende. Assemblés, trois graphiques font un tableau de bord.



Un tableau de bord rassemble trois ou quatre graphiques qui répondent aux questions d'un client.

```
fig = px.scatter(tips, x="total_bill", y="tip", color="day", size="size")
fig.show()
```

- ET STREAMLIT ?

Streamlit transforme un script Python en page web avec des menus et des boutons, sans écrire de HTML. Il ne tourne pas directement dans Colab, car il lui faut un serveur web : sur ton ordinateur, on l'installe avec `pip install streamlit`, on écrit `app.py`, puis on lance `streamlit run app.py`. À chaque clic, Streamlit relance le script et met la page à jour. Le notebook en montre un exemple à lire.

Raconter en deux minutes

Ton client n'a pas le temps : trois graphiques, deux minutes, trois temps.

TEMPS	CE QUE TU DIS	SUR TIPS
Constat, 20 s	Ce que tu as trouvé, en une phrase avec un chiffre	Le dimanche soir, le pourboire moyen atteint 3,26 \$, près de 9 % au-dessus de la moyenne.
Preuve, 60 s	Le graphique qui le montre, et comment le lire	Les barres du pourboire moyen par jour et par moment.
Recommandation, 40 s	Ce que le client peut faire demain	Mettre les meilleurs serveurs le dimanche soir.

À toi : ton dashboard et ton pitch

Choisis ton client : le patron du restaurant avec Tips, le créateur d'un jeu avec les Pokémon, ou quelqu'un d'autre avec ton dataset de la séance 2. Tu as 20 minutes.

Trois questions précises qui l'intéressent, et pour chacune un graphique Plotly dont le titre donne la réponse.

Un tableau de bord qui les assemble, puis la vérification des trois pièges : axe à zéro, cause ou corrélation, échantillon.

Ton pitch en trois temps, prêt à être dit à voix haute.

Entre deux séances

Le carnet `05_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées. Il ajoute un troisième jeu de données, le catalogue Netflix, et finit par un pitch pour son responsable.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. La corrélation entre la défense et la vitesse des Pokémon vaut 0,02. Qu'est-ce que ça veut dire ?
 - A. Les Pokémon rapides sont fragiles
 - B. La vitesse ne dit presque rien de la défense
 - C. La défense fait baisser la vitesse
 - D. Il y a une erreur dans les données
2. Blissey a 255 points de vie, bien au-delà de la limite de 125 de la boîte à moustaches. Que fais-tu de cette ligne ?
 - A. Je la supprime, c'est une erreur
 - B. Je la remplace par la médiane
 - C. Je la regarde de près : c'est peut-être la chose à raconter
 - D. Je la cache dans le graphique
3. Quel titre donner au nuage « pourboire selon l'addition » dans ton tableau de bord ?
 - A. Pourboire vs addition
 - B. Graphique 1
 - C. Le pourboire suit l'addition
 - D. Données Tips, colonnes total_bill et tip

4. Dans ton pitch, « mettre les meilleurs serveurs le dimanche soir », c'est quel temps ?

- A. Le constat
- B. La preuve
- C. La recommandation
- D. La question

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Une analyse commence par des questions précises, auxquelles les colonnes du tableau peuvent répondre.
2. Tendances, nuage de points et droite ; anomalie, boîte à moustaches ; corrélation, corr et heatmap.
3. Une corrélation dit que deux choses bougent ensemble, jamais que l'une cause l'autre.
4. Un axe tronqué invente un écart, un échantillon biaisé donne un chiffre précis et faux.
5. Un tableau de bord répond aux questions d'un client ; le pitch dit le constat, la preuve, la recommandation.

Corrigé

Quel piège ?

1. **Causalité** · Portable et bonnes notes vont ensemble, mais un troisième facteur peut produire les deux : le revenu du foyer, un endroit calme pour travailler. Distribuer des portables ne changerait pas ce facteur.
2. **Graphique trompeur** · De 4,1 à 4,3, la note gagne moins de 5 %. Une barre qui double veut dire que l'axe démarre juste sous 4 : c'est le dessin qui fabrique l'écart.
3. **Biais** · Ceux qui ont abandonné, sans doute les moins contents, n'ont jamais eu la parole. Le chiffre est exact pour ce groupe-là, faux pour l'ensemble des joueurs.
4. **Causalité** · Une explication possible, cachée : des parents myopes laissent plus volontiers une lumière, et transmettent aussi leur myopie. Veilleuse et myopie peuvent venir de la même cause.
5. **Biais** · Ceux qui fuient la cantine ne sont pas là pour répondre, et le jour choisi flatte le menu. On apprend l'avis des habitués un bon jour, pas celui des élèves.
6. **Graphique trompeur** · Les deux parts valent autant ; la perspective agrandit ce qui est devant. Les données sont justes, c'est le dessin qui ment.

Mini-quiz

1. **B. La vitesse ne dit presque rien de la défense** · Une corrélation proche de 0 veut dire qu'aucune direction ne se dégage : on trouve de tout, des lents solides, des rapides solides, des lents fragiles. Pour lire « les rapides sont fragiles », il faudrait une corrélation nettement négative.
2. **C. Je la regarde de près : c'est peut-être la chose à raconter** · La règle des 1,5 écarts signale un suspect, pas un coupable. Blissey a vraiment 255 HP dans le jeu : une anomalie vraie est souvent le fait le plus intéressant du tableau.
3. **C. Le pourboire suit l'addition** · Un titre de tableau de bord donne la réponse, pas le nom des colonnes : le client lit la conclusion avant même de regarder les points. Les autres titres l'obligent à tout décoder lui-même.
4. **C. La recommandation** · Le constat dit ce que tu as trouvé, avec un chiffre ; la preuve montre le graphique qui l'établit ; la recommandation dit au client quoi faire demain. Une action à mener, c'est toujours la recommandation.

Premier modèle

L'apprentissage supervisé sur 333 manchots : X et y, cacher une partie des données, lire un arbre de décision, reconnaître un modèle qui a appris par cœur, puis la même recette sur les Pokémon légendaires.

À LA FIN DE LA SÉANCE

Tu sauras dire ce que sont X et y, et ce que font fit et predict.

Tu sauras couper train et test, mesurer l'exactitude et la comparer au modèle bête.

Tu sauras lire un arbre de décision et reconnaître un modèle qui a appris par cœur.

Apprendre avec la réponse

Trois espèces de manchots vivent en Antarctique, et des scientifiques en ont mesuré 344. À partir des mesures seules, une machine peut-elle deviner l'espèce ?

1. des exemples avec la réponse

trois des 333 manchots mesurés

bec	épaisseur	nageoire	poids	espèce
39,1	18,7	181	3750	Adélie
46,5	17,9	192	3500	Chinstrap
50	15,3	220	5550	Gentoo

X

ce que la machine voit

y

ce qu'elle doit deviner

fit

2. une règle

trouvée par la machine dans les exemples,
pas écrite par toi

predict
→ Gentoo

3. un manchot sans réponse

45	15	215	5000	?
----	----	-----	------	---

Des exemples avec la réponse, une règle trouvée par la machine, une réponse devinée pour du nouveau.

```
arbre.fit(X_train, y_train)      # apprendre
predictions = arbre.predict(X_test) # deviner
```

Cacher une partie des données

Si on te donne les réponses de l'examen la veille, ta note ne prouve rien. Un modèle se juge sur des données qu'il n'a jamais vues.

1. on coupe avant d'apprendre

249 manchots pour apprendre (train, 75 %)

84 cachés (test, 25 %)

le modèle voit X et y des 249 ; des 84 cachés, il ne voit que X, et on compare ses réponses au vrai y

2. l'exactitude sur les 84 cachés

modèle bête : toujours « Adélie »

47,6 %

k plus proches voisins, k = 5

85,7 %

arbre de décision, profondeur 3

97,6 %

la barre à battre

On cache des données pour vérifier que le modèle a compris, et on compare toujours au modèle bête.

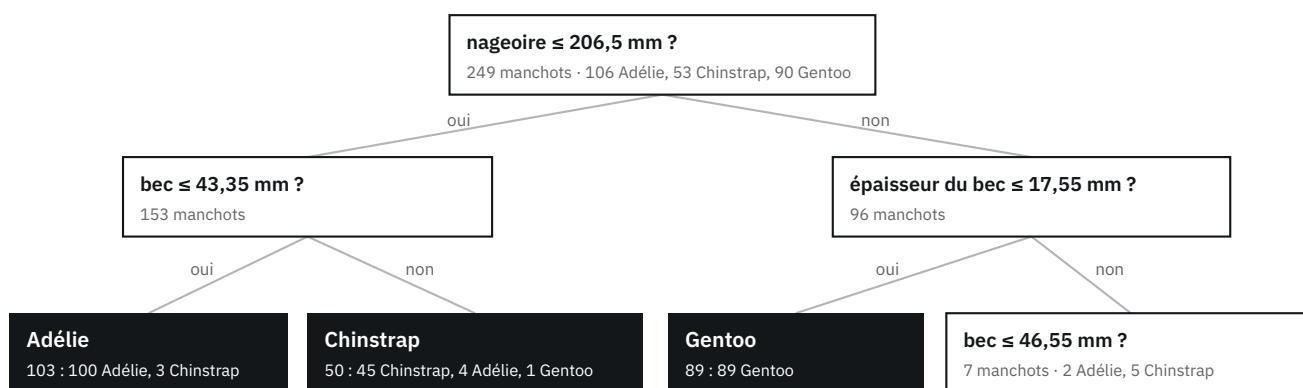
- EXACTITUDE, PAS PRÉCISION

L'exactitude, *accuracy* dans scikit-learn, est la part de bonnes réponses. En français courant on dirait « précision », mais en machine learning la précision est une autre mesure : parmi ce que le modèle a déclaré positif, la part qui l'est vraiment. Tu la retrouveras dans l'aide-mémoire scikit-learn du cours.

Pour deviner un nombre, bonne ou mauvaise réponse n'a plus de sens : on mesure de combien on se trompe en moyenne. La régression du poids des manchots se trompe de 296 g, le modèle bête qui répond toujours le poids moyen de 658 g.

Lire un arbre

Un arbre de décision pose des questions en cascade, comme au jeu « Qui est-ce ? ». Sur les 84 manchots cachés, il n'en rate que deux, des Chinstrap pris pour des Adélie.

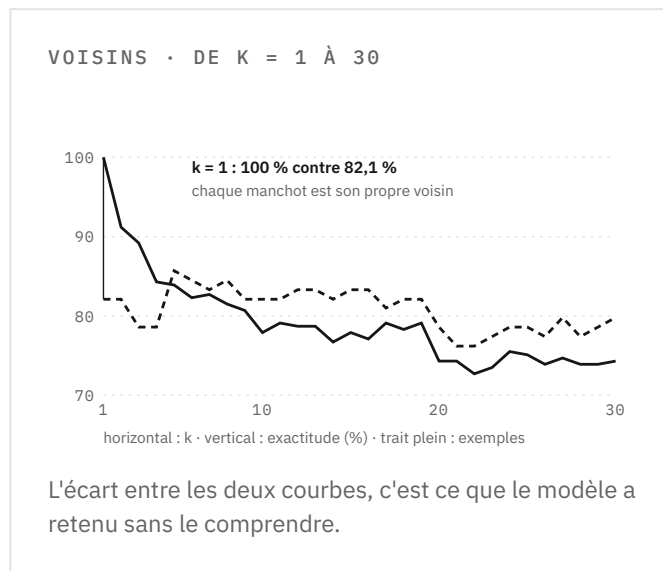
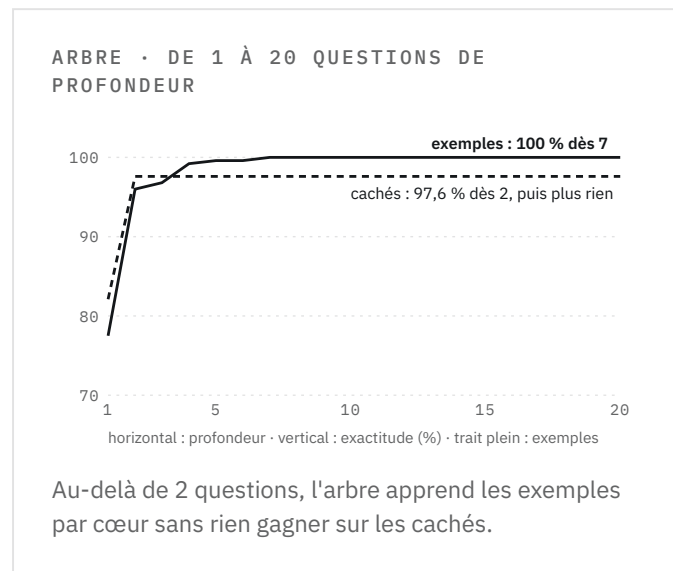


Chaque case dit combien de manchots d'entraînement y arrivent. Une feuille pleine donne l'espèce majoritaire.

Un arbre pose des questions en cascade, et on peut le lire à voix haute.

Appris par cœur

Laisse l'arbre poser autant de questions qu'il veut : il finit par connaître chaque manchot d'entraînement, sans faire mieux sur les autres.



CE MODÈLE A-T-IL COMPRIS ? • SUR PAPIER

Pour chaque modèle, tranche : a-t-il compris, appris par cœur, ou ne fait-il pas mieux que le modèle bête ?

Il a compris : sur les cachés, il fait nettement mieux que le modèle bête, et presque aussi bien que sur les exemples

Il a appris par cœur : il est parfait ou presque sur les exemples, nettement moins bon sur les cachés : c'est le sur-apprentissage

Pas mieux que le modèle bête : sur les cachés, il ne fait pas mieux que le modèle bête, quel que soit son score

1. Arbre de profondeur 3, manchots : 96,8 % sur les exemples, 97,6 % sur les cachés ; le modèle bête fait 47,6 %.
2. k-NN avec un seul voisin, manchots : 100 % sur les exemples, 82,1 % sur les cachés.
3. Arbre de profondeur 3, Pokémon légendaires : 92,0 % sur les cachés ; « jamais légendaire » fait aussi 92,0 %.
4. Régression du poids : 316 g d'erreur moyenne sur les exemples, 296 g sur les cachés ; le modèle bête se trompe de 658 g.
5. Arbre de régression sans limite de profondeur : 0 g d'erreur sur les exemples, 325 g sur les cachés.
6. k-NN qui consulte les 249 manchots d'entraînement comme voisins : 47,6 % sur les cachés.

Les réponses sont dans le corrigé, en fin de leçon.

À toi : les Pokémon légendaires

La même recette sur les 800 Pokémon, en 15 minutes : prédire `Legendary` à partir des six stats.

Piège : il n'y a que 65 légendaires, et répondre « jamais légendaire » à tout le monde fait déjà 91,9 %.

La barre à battre : le modèle bête sur tes données de test, puis un arbre de profondeur 3 et son tableau croisé.

Compte les légendaires retrouvés, pas seulement l'exactitude : c'est là que les modèles se départagent.

Ta phrase, lue dans l'arbre : « mon modèle a compris que... ».

Entre deux séances

Le carnet `06_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées. Il s'ouvre par un quiz de quatre questions, à se poser à voix haute au début de la séance suivante.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Dans le tableau des manchots, quelle colonne est y ?
 - A. La longueur du bec
 - B. Le poids
 - C. L'espèce
 - D. Toutes les mesures ensemble
2. Un manchot a une nageoire de 215 mm et un bec épais de 15 mm. Que répond l'arbre ?
 - A. Adélie
 - B. Chinstrap
 - C. Gentoo
 - D. L'arbre ne peut pas répondre
3. Le modèle répond juste sur 97,6 % des manchots cachés. Comment s'appelle ce pourcentage ?
 - A. La précision
 - B. L'exactitude
 - C. L'erreur moyenne
 - D. Le rappel

4. Les k plus proches voisins passent de 85,7 % à 98,8 % quand on met les mesures à la même échelle. Pourquoi ?
- A. Le modèle a plus de données
 - B. Le poids, en milliers de grammes, écrasait les longueurs en millimètres dans le calcul des distances
 - C. On a changé la coupe train / test
 - D. k a changé

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Apprentissage supervisé, des exemples avec la réponse, la machine trouve la règle et l'applique à du nouveau.
2. X, ce que la machine voit ; y, ce qu'elle doit deviner ; fit apprend, predict devine.
3. On cache un quart des données pour vérifier, et on compare toujours au modèle bête.
4. L'exactitude est la part de bonnes réponses ; pour un nombre, on mesure l'erreur moyenne.
5. Parfait sur les exemples, moins bon sur les cachés, c'est le sur-apprentissage.

Corrigé

Ce modèle a-t-il compris ?

1. **Il a compris** · Presque le même score des deux côtés, et deux fois celui du modèle bête : la règle apprise vaut aussi pour des manchots jamais vus.
2. **Il a appris par cœur** · Avec un seul voisin, chaque manchot d'entraînement se reconnaît lui-même : 100 % garanti. Les 18 points perdus sur les cachés mesurent ce qu'il a retenu sans le comprendre.
3. **Pas mieux que le modèle bête** · 92 % a l'air excellent, mais répondre « non » à tout le monde fait pareil. Le tableau croisé le confirme : 3 légendaires retrouvés sur 16.
4. **Il a compris** · L'erreur est la même des deux côtés et deux fois plus petite que celle du modèle bête, qui répond toujours le poids moyen.
5. **Il a appris par cœur** · Zéro gramme d'erreur, c'est une feuille par manchot : l'arbre a recopié les exemples. Sur les cachés, il fait moins bien que la simple régression linéaire.
6. **Pas mieux que le modèle bête** · Avec tout le monde pour voisin, la majorité est toujours Adélie : il répond Adélie à chaque fois, exactement comme le modèle bête.

Mini-quiz

1. **C. L'espèce** · y, c'est ce que la machine doit deviner : l'espèce. Les quatre mesures forment X, ce qu'elle voit. Pour la régression du poids, les rôles changent : le poids devient y.
2. **C. Gentoo** · 215 mm, c'est plus que 206,5 : on part à droite. Puis 15 mm d'épaisseur, c'est moins que 17,55 : la feuille Gentoo, où tombent 89 manchots d'entraînement, tous Gentoo.
3. **B. L'exactitude** · La part de bonnes réponses s'appelle l'exactitude, accuracy en anglais. La précision existe aussi en machine learning, mais elle mesure autre chose ; l'erreur moyenne, elle, sert aux nombres.
4. **B. Le poids, en milliers de grammes, écrasait les longueurs en millimètres dans le calcul des distances** · k-NN compare des distances. Sans mise à l'échelle, 100 g d'écart pèsent autant que 100 mm, alors que les becs ne diffèrent que de quelques millimètres : le poids décidait presque seul.

Kaggle Titanic 1/2

Le grand projet du parcours, en binôme : explorer les 891 passagers du Titanic, vérifier quatre hypothèses, créer des variables dont le titre extrait du nom, mesurer deux modèles par validation croisée et faire sa première soumission.

À LA FIN DE LA SÉANCE

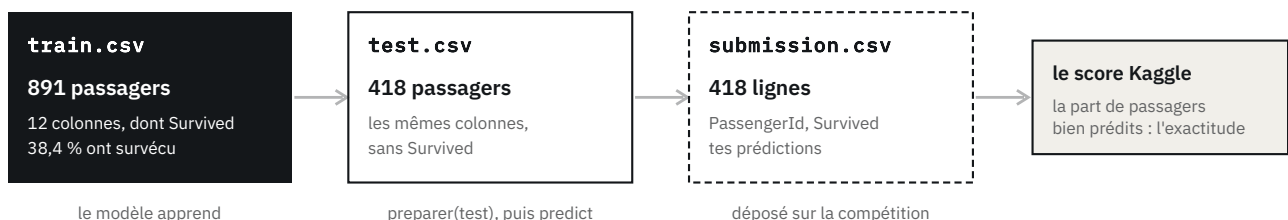
Tu sauras vérifier une hypothèse avec un taux de survie et un graphique, avant de modéliser.

Tu sauras créer une variable, dont un titre extrait d'un texte avec une regex, et ranger la préparation dans une fonction.

Tu sauras mesurer un modèle par validation croisée et soumettre tes prédictions à une compétition.

Une compétition, trois fichiers

Le 15 avril 1912, le Titanic coule : 1 502 des 2 224 passagers et membres d'équipage meurent. Kaggle donne la fiche de 891 passagers et leur sort ; à toi de prédire celui de 418 autres.



La réponse des 418 passagers de test, Kaggle la garde : c'est ce qui rend le classement honnête.

Apprendre sur train, prédire test, soumettre deux colonnes, recevoir un score.

– AVANT LA SÉANCE : LES FICHIERS

Chacun crée un compte Kaggle gratuit, rejoint la compétition Titanic, qui ne demande pas d'accepter de règlement, et télécharge `train.csv` et `test.csv` ; le notebook détaille chaque étape. Dans Colab, on les dépose dans le panneau des fichiers, et ils disparaissent quand la session se ferme. Sans eux, le notebook charge une copie publique de `train.csv` : tout fonctionne, sauf la soumission.

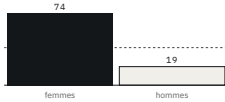
La page de la compétition ↗

Quatre hypothèses

Un data scientist ne devine pas, il vérifie. Le trait pointillé marque la survie moyenne, 38,4 %.

HYPOTHÈSE 1 · CONFIRMÉE

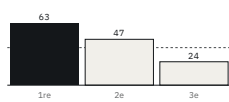
Les femmes et les enfants d'abord



74 % des femmes survivent, 19 % des hommes.

HYPOTHÈSE 2 · CONFIRMÉE

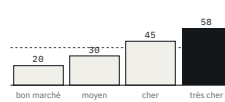
La classe compte



63 % en 1re classe, 24 % en 3e.

HYPOTHÈSE 3 · CONFIRMÉE

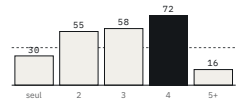
Le prix du billet



près de 3 fois plus de survivants chez les plus chers.

HYPOTHÈSE 4 · NUANCÉE

La famille à bord



mieux à 2, 3 ou 4 ; seul ou à 5 et plus, c'est pire.

QUI A PLUS DE CHANCES ? · SUR PAPIER

Pour chaque duel, dis quel passager avait le plus de chances de survivre.

1. une femme de 3e classe ou un homme de 1re classe ?
2. un passager seul ou un passager d'une famille de 4 ?
3. un passager d'une famille de 6 ou un passager seul ?
4. un garçon au titre « Master » ou un homme au titre « Mr » ?
5. un passager embarqué à Southampton ou un passager embarqué à Cherbourg ?
6. un enfant de moins de 12 ans ou une femme ?

Les réponses sont dans le corrigé, en fin de leçon.

Nettoyer et créer des variables

177 âges manquent, soit un passager sur cinq. Plutôt que de remplir tous les trous avec le même âge, le nom du passager en dit plus qu'on ne croit.

1. le titre, entre la virgule et le point

Braund, Mr. Owen Harris

,\s*([^\.]+)\.

une virgule, des espaces éventuels,
puis tout ce qui n'est pas un point : on le garde

Mr

17 titres différents, dont beaucoup vus une fois
(Capt, Jonkheer...) : on les regroupe en cinq

2. ce que dit le titre

titre	passagers	survie	âge médian
Mr	517	16 %	30 ans
Miss	185	70 %	21 ans
Mrs	126	79 %	35 ans
Master	40	57 %	3,5 ans
Autre	23	35 %	48,5 ans

Pour les 177 âges manquants : l'âge médian du titre (3,5 ans pour un Master)
plutôt que 28 ans pour tout le monde, qui ferait un pic artificiel.

Le titre dit d'un coup le sexe, l'âge et le rang : l'une des variables les plus fortes du Titanic.

```
train["Titre"] = train["Name"].str.extract(r",\s*([^\.]+)\.")[0]
```

Une recette, cinq contrôles

Tout le nettoyage tient dans une fonction, `preparer()`, appliquée telle quelle à `train` puis à `test`.
Pour mesurer, on ne coupe plus une seule fois : on fait passer cinq contrôles.

1. cinq contrôles blancs sur les 891 passagers

contrôle					85,5 %
	contrôle				82,0 %
		contrôle			82,6 %
			contrôle		79,8 %
				contrôle	83,7 %

clair : les quatre paquets où la forêt apprend · plein : le paquet du contrôle

moyenne 82,7 %

2. la moyenne des cinq contrôles

modèle bête : personne ne survit

61,6 %

régression logistique

80,4 %

forêt, 200 arbres, profondeur 5

82,7 %

Cinq contrôles avec des sujets différents valent mieux qu'un seul.

- DEUX NOUVEAUX MODÈLES

La régression logistique est une cousine de la régression linéaire de la séance 6 : elle pèse chaque indice, puis bascule vers « survit » ou « ne survit pas ». La forêt aléatoire réunit 200 arbres de décision, chacun appris sur un échantillon un peu différent, qui votent. Sans limite de profondeur, la forêt tombe à 80,5 % : ses arbres apprennent par cœur, ce que la séance 8 reprend.

À toi : ta première soumission

En binôme, 10 minutes : l'un tape, l'autre réfléchit à voix haute.

Une variable de plus dans `preparer()` , par exemple le prix par personne, et la forêt remesurée.

Génère `submission.csv` avec la meilleure forêt, dépose-le sur la page de la compétition, et note le score dans le notebook.

Si le temps manque, la soumission ouvre la séance 8.

Entre deux séances

Le carnet `07_exercices.ipynb` : 12 exercices, de ★ à ★★★★★ , avec vérification automatique, indices et solutions repliées. Il s'ouvre par un quiz de deux questions, à se poser à voix haute au début de la séance suivante.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Pourquoi ranger tout le nettoyage dans une seule fonction `preparer()` ?
 - A. Pour que le code soit plus court
 - B. Pour appliquer exactement la même recette à train et à test
 - C. Parce que Kaggle l'exige
 - D. Pour aller plus vite
2. Que garde la regex `,s*([^\.,]+)\.` dans le nom « Heikkinen, Miss. Laina » ?
 - A. Heikkinen
 - B. Miss
 - C. Laina
 - D. Miss. Laina
3. Les cinq contrôles de la forêt donnent de 79,8 % à 85,5 %. Quel score annonces-tu ?
 - A. 85,5 %, le meilleur
 - B. 79,8 %, le pire
 - C. 82,7 %, la moyenne des cinq
 - D. Celui du premier contrôle

4. Pourquoi remplir les âges manquants avec l'âge médian du titre plutôt qu'avec 28 ans pour tout le monde ?
- A. Parce que 28 ans est faux
 - B. Parce qu'un petit garçon au titre « Master » recevrait 28 ans, et 177 passagers formeraient un pic artificiel
 - C. Parce que la médiane est interdite
 - D. Pour avoir moins de lignes

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Sur Kaggle, train.csv sert à apprendre, test.csv à prédire, et le score est la part de passagers bien prédits.
2. On vérifie ses hypothèses avant de modéliser, chacune avec un chiffre.
3. Une nouvelle variable bien choisie, comme le titre, vaut souvent mieux qu'un modèle plus compliqué.
4. Une seule fonction `preparer()` pour train et test, sinon les deux fichiers ne se ressemblent plus.
5. La validation croisée fait passer cinq contrôles au modèle et garde la moyenne.

Corrigé

Qui a plus de chances ?

1. **une femme de 3e classe : 50 % de survie, contre 37 %** · Le sexe pèse plus lourd que la classe : même en 3e, une femme avait plus de chances qu'un homme de 1re.
2. **un passager d'une famille de 4 : 72 % de survie, contre 30 %** · Une petite famille a sans doute pu s'entraider. Mais attention, 29 passagers seulement : un petit groupe fait des taux fragiles.
3. **un passager seul : 30 % de survie, contre 14 %** · Au-delà de 4, la survie s'effondre. Ces grandes familles voyageaient surtout en 3e classe : 17 passagers sur 22 ici.
4. **un garçon au titre « Master » : 57 % de survie, contre 16 %** · « Master » désignait les petits garçons (âge médian 3,5 ans) : enfants d'abord. Le titre dit à la fois le sexe et l'âge.
5. **un passager embarqué à Cherbourg : 55 % de survie, contre 34 %** · Le port ne sauve personne : à Cherbourg, 85 passagers sur 168 étaient en 1re classe. C'est la classe qui se cache derrière le port.
6. **une femme : 74 % de survie, contre 57 %** · Les enfants s'en sortent mieux que la moyenne, mais les femmes plus encore : beaucoup d'enfants voyageaient en 3e classe.

Mini-quiz

1. **B. Pour appliquer exactement la même recette à train et à test** · Le modèle apprend sur des colonnes préparées d'une certaine façon ; test doit arriver exactement pareil. L'erreur classique : nettoyer train à la main et oublier une étape sur test.
2. **B. Miss** · Elle cherche une virgule, d'éventuels espaces, puis garde tout ce qui n'est pas un point, jusqu'au point : « Miss ». Les parenthèses marquent ce qu'on garde.
3. **C. 82,7 %, la moyenne des cinq** · Chaque contrôle dépend du paquet tiré ; la moyenne des cinq est plus fiable qu'un seul. Annoncer le meilleur, ce serait choisir le sujet d'examen qui t'arrange.
4. **B. Parce qu'un petit garçon au titre « Master » recevrait 28 ans, et 177 passagers formeraient un pic artificiel** · 28 ans est bien l'âge médian du bateau, mais pas celui de chacun. Le titre donne un âge plausible : 3,5 ans pour un Master, 35 pour une Mrs.

Kaggle Titanic 2/2

Passer en mode compétition : comparer quatre modèles, comprendre pourquoi l'un gagne, voir le sur-apprentissage, régler deux boutons avec GridSearchCV, déjouer les pièges, puis soumettre et présenter sa démarche en cinq minutes.

À LA FIN DE LA SÉANCE

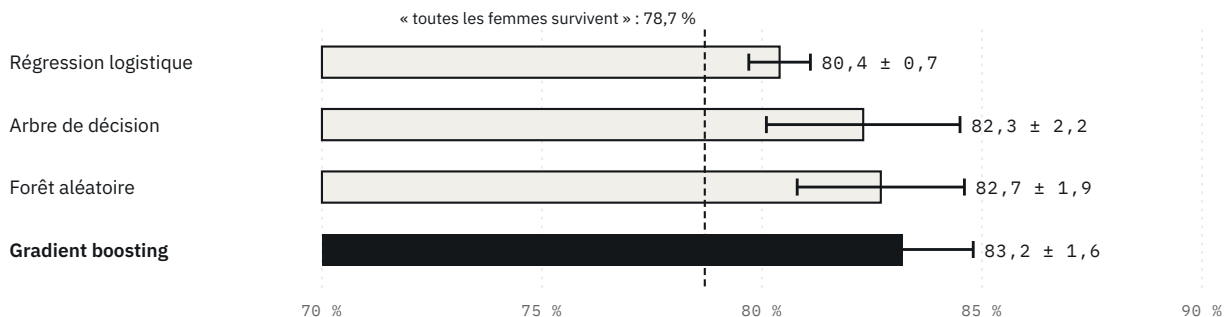
Tu sauras comparer des modèles avec la même validation croisée, en lisant l'écart-type autant que la moyenne.

Tu sauras expliquer une décision par l'importance des variables, et régler un modèle avec GridSearchCV.

Tu sauras reconnaître une fuite de données et un score trop beau pour être vrai.

Quatre modèles, un tableau

On reprend `preparer()` de la séance 7 et on mesure quatre familles de modèles avec la même validation croisée.



Le trait noir sur chaque barre : l'écart-type des cinq paquets. Un écart de 1 point entre deux modèles peut n'être que du hasard.

Quatre modèles dans un mouchoir de poche, et tous au-dessus de la règle bête.

– LE GRADIENT BOOSTING

Le gradient boosting construit de petits arbres l'un après l'autre, chacun corrigeant les erreurs du précédent : comme réviser en refaisant surtout les exercices ratés. La forêt, elle, fait voter des arbres construits chacun de son côté. Réglé par GridSearchCV, le boosting monte à 84,1 %.

Pourquoi l'un gagne



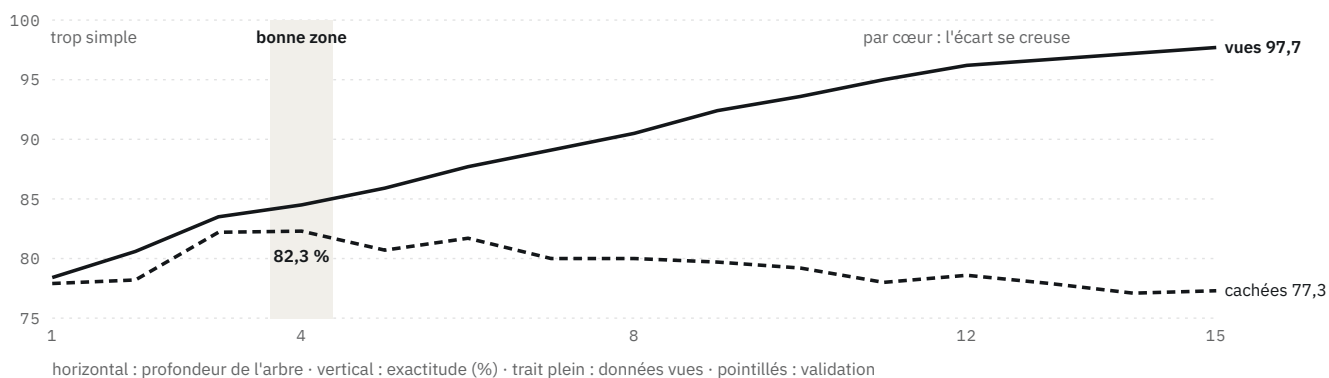
La forêt partage entre Titre et Sex, qui racontent la même histoire ; le boosting mise presque la moitié sur Titre.

Un score sans explication ne convainc personne : l'importance des variables dit sur quoi le modèle décide.

Retire Titre : la forêt passe de 82,7 à 82,0 % seulement, car Sex et Age portent presque la même information.

Apprendre par cœur

Comme l'élève qui connaît les annales par cœur et sèche devant un sujet nouveau.



Au-delà de la bonne profondeur, l'arbre gagne sur ce qu'il a vu et perd sur ce qu'il n'a pas vu.

Régler les boutons

		min_samples_leaf : passagers minimum par feuille			4 × 3 = 12 combinaisons × 5 paquets = 60 entraînements meilleur réglage : 82,9 % profondeur 5, feuilles d'au moins 3 tous entre 82,0 et 82,9 % : régler aide, mais moins qu'une bonne variable
		1	3	5	
max_depth	3	82,5 %	82,6 %	82,4 %	
	5	82,3 %	82,9 %	82,8 %	
	7	82,5 %	82,0 %	82,6 %	
	9	82,6 %	82,7 %	82,5 %	

Un hyperparamètre se choisit avant l'entraînement ; GridSearchCV essaie toutes les combinaisons d'une grille.

```
grille = {"max_depth": [3, 5, 7, 9], "min_samples_leaf": [1, 3, 5]}
foret = RandomForestClassifier(n_estimators=100, random_state=42)
recherche = GridSearchCV(foret, grille, cv=5).fit(X, y)
```

Trois pièges

Des scores magnifiques, et faux.

PIÈGE 1 · LA FUITE DE
DONNÉES

100,0 %

Une colonne Canot qui recopie la survie : le modèle triche sans le savoir.

Sur test.csv, la colonne n'existe pas : le modèle est inutilisable.

PIÈGE 2 · SE NOTER SUR
SES DONNÉES

98,1 %

Une forêt sans limite, notée sur les passagers qu'elle a vus.

En validation croisée, la même forêt fait 80,5 %.

PIÈGE 3 · OUBLIER LA
RÈGLE BÊTE

0,76555

« Toutes les femmes survivent » : 320 bonnes réponses sur 418 sur Kaggle.

Un modèle qui ne bat pas ce score ne sert à rien.

FUITE OU PAS FUITE ? · SUR PAPIER

Pour chaque colonne ajoutée, dis si elle fait fuiter la réponse.

Fuite : la colonne contient la réponse, ou n'existe qu'une fois la réponse connue : impossible de la calculer pour un passager de test.csv

Pas de fuite : l'information existait avant le naufrage, ou se calcule à partir de train seul, pour n'importe quel passager de test.csv

1. On ajoute une colonne Canot qui vaut 1 pour les passagers montés dans un canot.
2. On remplit les âges manquants de test.csv avec la médiane calculée sur train.
3. On ajoute le nombre de survivants parmi les passagers qui ont le même numéro de billet.
4. On ajoute le titre, Mr, Mrs, Miss, Master, extrait du nom.
5. On ajoute la taille de la famille à bord, SibSp + Parch + 1.
6. On ajoute le numéro d'identification des corps retrouvés après le naufrage.

Les réponses sont dans le corrigé, en fin de leçon.

- DES 1,00000 AU CLASSEMENT

Le classement de la compétition, calculé sur les 418 passagers de test, affiche des équipes à 1,00000. Personne ne prédit parfaitement un naufrage : la liste réelle des survivants est publique, et ces scores la recopient sans doute. Ce n'est pas du machine learning, et ça n'apprend rien.

À toi : la soumission finale

En binôme, 20 minutes, dans l'ordre :

Le modèle final : la forêt réglée par GridSearchCV ou le boosting, remesuré après une variable ajoutée ou retirée.

submission_finale.csv , soumis sur Kaggle, et tes deux scores notés : première soumission, finale.

Le tableau du groupe : chaque binôme annonce ses scores et son modèle.

Présenter en cinq minutes

Pas de diapositives obligatoires : le notebook et un ou deux graphiques suffisent.

PARTIE	DURÉE	CONTENU
Nos hypothèses	1 min	Les deux qui vous ont marqués, avec un chiffre chacune
Ce qu'on a essayé	2 min	Variables, modèles, le réglage qui a le plus compté, une chose qui n'a pas marché
Notre score	1 min	Première soumission, finale, et la place face à la règle bête
Ce qu'on referait	1 min	Avec deux heures de plus : quelle variable, quel modèle, quel piège éviter

Entre deux séances

Le carnet 08_exercices.ipynb : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées. Il s'ouvre par une question, à se poser à voix haute au début de la séance suivante.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. L'arbre fait $82,3 \% \pm 2,2$, la régression logistique $80,4 \% \pm 0,7$. Lequel est le plus stable d'un paquet à l'autre ?
 - A. L'arbre, parce que sa moyenne est plus haute
 - B. La régression logistique, parce que son écart-type est plus petit
 - C. Les deux se valent
 - D. On ne peut pas savoir
2. Sans la variable Titre, la forêt passe seulement de $82,7 \%$ à $82,0 \%$. Pourquoi si peu ?
 - A. Titre ne servait à rien
 - B. Sex et Age portent presque la même information que Titre
 - C. La forêt a ignoré la consigne
 - D. Le score est faux
3. Lequel de ces réglages est un hyperparamètre ?
 - A. Le seuil « nageoire $\leq 206,5$ mm » trouvé par un arbre
 - B. max_depth, la profondeur maximale de l'arbre
 - C. Le score de validation croisée
 - D. La colonne Survived
4. Une grille de 4 profondeurs $\times$ 3 tailles de feuille, avec une validation croisée à 5 paquets : combien d'entraînements ?
 - A. 12
 - B. 15
 - C. 20
 - D. 60

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Même validation croisée pour tous les modèles, et un écart d'un point peut n'être que du hasard.

2. L'importance des variables dit sur quoi le modèle décide ; c'est l'argument à montrer.
3. Train qui monte, validation qui descend, c'est le sur-apprentissage.
4. Un hyperparamètre se règle avant l'entraînement ; GridSearchCV essaie toute une grille.
5. Un score trop beau est une alarme, fuite ou mesure sur ses propres données, et on bat toujours la règle bête d'abord.

Corrigé

Fuite ou pas fuite ?

1. **Fuite** · Être monté dans un canot, c'est presque avoir survécu. Le notebook le montre : avec la réponse déguisée, la forêt fait 100 %, et test.csv n'a pas cette colonne.
2. **Pas de fuite** · La médiane ne dépend que des passagers d'entraînement, et elle s'applique à n'importe qui. C'est même la bonne pratique : ne jamais regarder test pour préparer.
3. **Fuite** · Pour fabriquer cette colonne, il faut connaître Survived. Sur train le score s'envole ; sur test.csv, la colonne ne se calcule pas.
4. **Pas de fuite** · Le nom figure sur le billet, bien avant le naufrage, et test.csv le contient. C'est une variable créée, pas une réponse cachée.
5. **Pas de fuite** · Deux colonnes connues à l'embarquement, présentes dans les deux fichiers : aucune trace de la survie dans le calcul.
6. **Fuite** · Cette information n'existe que pour les victimes, donc seulement une fois la réponse connue. Un modèle qui s'en sert triche sans le savoir.

Mini-quiz

1. **B. La régression logistique, parce que son écart-type est plus petit** · L'écart-type mesure combien le score varie d'un paquet à l'autre. Celui de la logistique bouge de moins d'un point, celui de l'arbre de plus de deux : sa moyenne est plus haute, mais moins sûre.
2. **B. Sex et Age portent presque la même information que Titre** · Un titre dit le sexe et, pour Master ou Miss, l'âge. Quand on le retire, le modèle retrouve l'essentiel dans les colonnes Sex et Age : une variable importante n'est pas forcément irremplaçable.
3. **B. max_depth, la profondeur maximale de l'arbre** · Un hyperparamètre se choisit avant l'entraînement, comme la profondeur maximale. Les seuils des questions, eux, le modèle les apprend tout seul pendant fit.
4. **D. 60** · 12 combinaisons, et chacune est entraînée 5 fois, une par paquet : 60 entraînements. C'est pour ça qu'une grille trop grande devient vite lente.

Comment fonctionne un LLM

Ouvrir le capot d'un modèle de langage : des tokens, un seul geste répété, deviner le mot suivant, la température qui règle le hasard, et un petit modèle qu'on fait halluciner exprès.

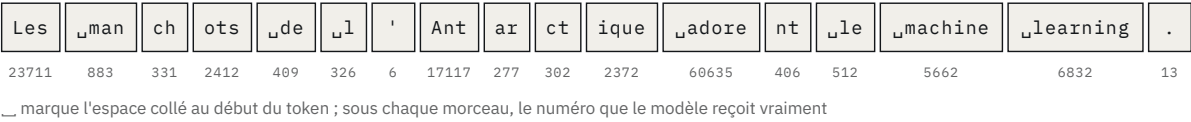
À LA FIN DE LA SÉANCE

- Tu sauras ce qu'est un token, et pourquoi un modèle compte mal les lettres.
- Tu sauras expliquer « prédire le mot suivant » et ce que change la température.
- Tu sauras reconnaître une hallucination, une date de connaissance dépassée et un calcul faux.

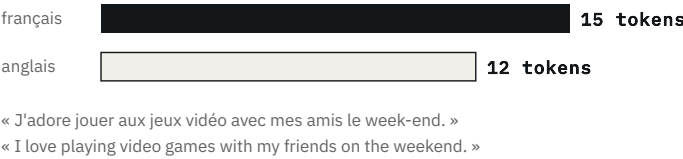
Des tokens, pas des lettres

Un ordinateur ne lit que des nombres. Avant de lire ta phrase, le modèle la découpe en tokens, et chaque token a un numéro. Tout ce qui suit sort du modèle du notebook, Qwen 2.5 0.5B, lancé pour de vrai.

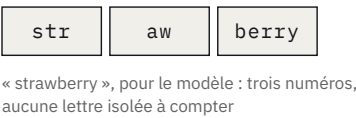
1. ce que lit le modèle : 17 tokens, pas des lettres



2. le même message, deux langues



3. pourquoi il compte mal les lettres



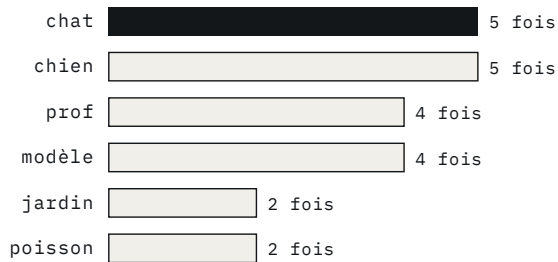
Un modèle lit des morceaux de mots numérotés ; le français en demande plus que l'anglais.

Deviner le mot suivant

Un modèle de langage ne fait qu'une chose : lire ce qui précède et deviner le token suivant, puis recommencer. Le notebook construit d'abord le plus petit modèle possible, un bigramme.

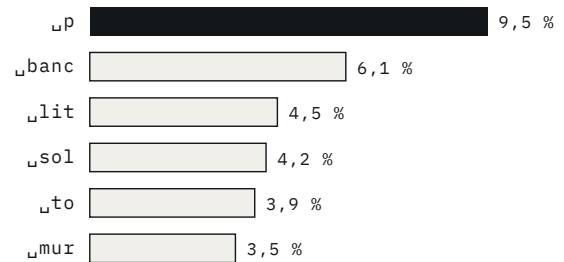
le bigramme : après « le », il a vu...

36 phrases, un comptage par couple de mots



le vrai modèle : après « Le chat dort sur le »...

probabilité de chaque token suivant, sur > 150 000 possibles



« ▮p », « ▮to » : des débuts de mots ; le mot se finit au token d'après

Le bigramme compte après un seul mot ; le vrai modèle calcule une probabilité après toute la phrase.

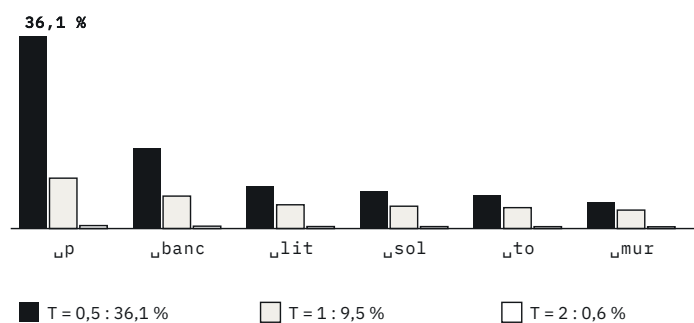
- POURQUOI LE BIGRAMME TOURNE EN ROND

Parti de « le », le bigramme choisit toujours le mot le plus fréquent : « chat », puis « dort », puis « dans », puis de nouveau « le ». Il écrit « le chat dort dans le chat dort dans le chat... » sans fin. Un vrai modèle regarde des milliers de tokens précédents, pas un seul, et il a lu des milliards de phrases, pas 36.

La température

Les vrais modèles tirent au sort parmi les tokens probables. La température règle ce hasard.

même phrase, trois températures



le pourcentage : la part du premier token, « ▮p », à chaque température ; softmax(logits ÷ T)

« un nom original pour un chat »

température 0

"Kitty"
"Kitty"
"Kitty"

température 1,5

Papillon
"ChatGym"
Chatbot

vrai modèle, graines 0, 1, 2

La température règle le hasard : basse, toujours la même réponse ; haute, des choix de plus en plus égaux.

Ce qu'il ne sait pas

Le modèle écrit le plus plausible, pas le vrai. Posées au vrai modèle, quatre questions précises :

LA QUESTION Qui a gagné la Coupe du monde de football 2031 ? LE MODÈLE RÉPOND La France a gagné la Coupe du Monde de football 2031 en remportant le titre. La vérité. Cette Coupe du monde n'a pas encore eu lieu.	LA QUESTION Combien font $4\,817 \times 2\,953$? Réponds juste le nombre. LE MODÈLE RÉPOND 24,605,311 La vérité. Python : 14 224 601	LA QUESTION Combien de lettres r dans le mot strawberry ? LE MODÈLE RÉPOND Le mot "strawberry" contient 4 lettres r. La vérité. Python : 3	LA QUESTION Qui est le maire de Trifouillis-les-Oies ? LE MODÈLE RÉPOND Le maire de Trifouillis-les-Oies est Jean-Pierre Rousset. La vérité. Ce village n'existe pas.
---	---	---	--

QUELLE LIMITE ? • SUR PAPIER

Pour chaque réponse du modèle, nomme la limite qui explique l'erreur.

Hallucination : il invente, sûr de lui, un fait sur quelque chose qui n'existe pas ou n'a pas encore eu lieu

Date de connaissance : il répond sur l'actualité avec des informations arrêtées à ses données d'entraînement

Calcul : il donne un résultat de calcul faux : il écrit des chiffres plausibles au lieu de poser l'opération

Lettres et tokens : il se trompe sur des lettres, parce qu'il voit des morceaux de mots, pas des lettres

1. « Qui a gagné Roland-Garros en 2034 ? » · réponse du modèle : « L'équipe de France a gagné Roland-Garros en 2034. »

2. « Résume en 2 phrases la carrière du chanteur Théo Bilborne. » · réponse du modèle : « Théo Bilborne a remporté le concours de rap "Rap de l'année" et est sorti avec "Bilborne". »

3. « Quel est le dernier iPhone sorti, et en quelle année sommes-nous ? » · réponse du modèle : « Le dernier iPhone a été sorti en 2019. Nous sommes actuellement en 2023. »

4. « Quel est le dernier jeu sorti sur Nintendo Switch ? » · réponse du modèle : « Le dernier jeu sorti sur Nintendo Switch était "Super Mario Odyssey". »

5. « Combien font $7\,391 \times 6\,128$? Réponds juste le nombre. » · réponse du modèle : « 40,596,320 »

6. « Combien de lettres e dans le mot anticonstitutionnellement ? » · réponse du modèle : « 7 »

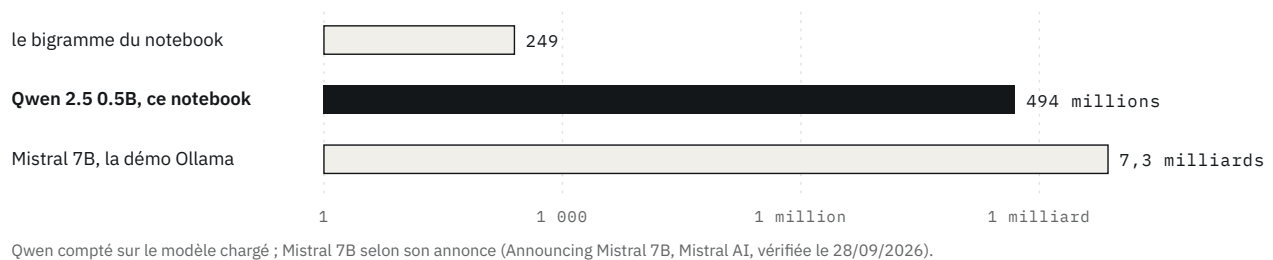
Les réponses sont dans le corrigé, en fin de leçon.

- ET LA CONSIGNE « SI TU N'ES PAS SÛR, DIS JE NE SAIS PAS » ?

Ajoutée au prompt système, elle n'a pas suffi au petit modèle : il a inventé un autre maire, avec une date d'élection. Plus le modèle est petit, moins il suit les consignes.

Petit modèle, gros modèle

Le modèle du cours tourne dans Colab gratuit, et même sur un ordinateur portable. Un gros modèle tourne sur les serveurs de son éditeur : il faut du réseau, et tes messages partent là-bas.



Un paramètre est un nombre réglé pendant l'entraînement ; le petit modèle du cours en a un demi-milliard.

- LE SAC DE MOTS, AVANT LES LLM

Avant les LLM, un filtre anti-spam comptait les mots d'un message : « gagné », « gratuit », « cliquez » sentent le spam. Le notebook refait cette méthode sur 20 messages, puis la piège. Résultat du vrai passage : le sac de mots laisse passer le faux message de la banque, faute d'en connaître les mots, et le petit modèle répond « spam » à tout, même à « On se voit demain au foot ? ». À cette taille, il suit mal la consigne.

À toi : fais halluciner le modèle

Tu deviens testeur de LLM, 20 minutes.

Cinq pièges, un par catégorie : un événement futur, une personne inventée, un calcul, l'actualité, des lettres à compter.

Note la réponse et la vérité, vérifiée avec Python ou une source fiable. Une bonne réponse est aussi une observation.

Ta fiche « 3 limites », chacune avec son explication et ton meilleur piège.

Entre deux séances

Le carnet 09_exercices.ipynb : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées. Il s'ouvre par un quiz de deux questions, à se poser à voix haute au début de la séance suivante.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. La même idée coûte 15 tokens en français et 12 en anglais. Pourquoi ?
 - A. Le français a plus de lettres
 - B. Le découpage a été construit sur beaucoup plus de texte anglais : ses gros morceaux sont surtout des mots anglais
 - C. Les accents comptent double
 - D. C'est un hasard
2. Parti de « le », le bigramme écrit « le chat dort dans le chat dort dans le chat... ». Pourquoi tourne-t-il en rond ?
 - A. Il manque des phrases
 - B. Il choisit toujours le mot le plus fréquent après le mot d'avant, et « dans » ramène toujours à « le »
 - C. Le mot « chat » est interdit
 - D. Il a trop de paramètres
3. Trois fois la même question à température 0 : que se passe-t-il ?
 - A. Trois réponses différentes
 - B. Trois fois la même réponse, le token le plus probable à chaque étape
 - C. Le modèle refuse de répondre
 - D. La réponse est plus juste
4. Avec le petit modèle du cours, dans Colab, où partent tes messages ?
 - A. Sur les serveurs d'une grande entreprise
 - B. Nulle part : le modèle tourne dans ta session, tes données restent là
 - C. Chez Hugging Face à chaque question
 - D. Sur ton compte Google Drive

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Un modèle lit des tokens, des morceaux de mots numérotés, pas des lettres.
2. Un LLM ne fait qu'une chose, prédire le token suivant, encore et encore.
3. La température règle le hasard, basse pour la même réponse, haute pour la variété puis le n'importe quoi.
4. Il écrit le plausible, pas le vrai ; plus la question est précise, plus il faut vérifier.
5. Petit modèle, gratuit et chez toi ; gros modèle, dans le cloud, meilleur et payant.

Corrigé

Quelle limite ?

1. **Hallucination** · 2034 n'a pas encore eu lieu, et aucune équipe nationale ne gagne Roland-Garros. Le modèle écrit la suite la plus plausible, pas la vraie.
2. **Hallucination** · Ce chanteur n'existe pas. Plutôt que « je ne sais pas », le modèle fabrique une carrière crédible.
3. **Date de connaissance** · Il ne sait pas quel jour on est : il répond avec ce qu'il a lu pendant son entraînement, déjà ancien.
4. **Date de connaissance** · Un jeu réel, mais pas le dernier : ses données s'arrêtent avant aujourd'hui, et il ne le dit pas.
5. **Calcul** · Python donne 45 292 048. Le modèle écrit un nombre qui a l'air juste, chiffre après chiffre, sans poser l'opération.
6. **Lettres et tokens** · Python compte 3 e. Le modèle voit ant, icon, stitution, nel, lement : des morceaux, pas des lettres.

Mini-quiz

1. **B. Le découpage a été construit sur beaucoup plus de texte anglais : ses gros morceaux sont surtout des mots anglais** · Un tokenizer garde en un seul morceau les suites de lettres qu'il a le plus vues. Un mot français est plus souvent coupé en deux ou trois : la même phrase coûte plus de tokens, donc plus de calcul.
2. **B. Il choisit toujours le mot le plus fréquent après le mot d'avant, et « dans » ramène toujours à « le »** · Un bigramme ne regarde qu'un mot en arrière. Toujours prendre le plus fréquent l'enferme dans une boucle ; tirer au sort parmi les mots probables, c'est ce que règle la température.
3. **B. Trois fois la même réponse, le token le plus probable à chaque étape** · À température 0, il n'y a plus de tirage : le vrai modèle a répondu « Kitty » les trois fois. Plus haut, les réponses varient, parfois jusqu'à l'absurde.
4. **B. Nulle part : le modèle tourne dans ta session, tes données restent là** · Le notebook télécharge le modèle une fois, puis tout se calcule dans ta session Colab. Avec l'API d'un gros modèle, au contraire, chaque message part sur le serveur de son éditeur.

Parler à un LLM par le code

Parler à un modèle depuis Python : une liste de messages, trois rôles, un prompt système qui donne une personnalité et des règles, un historique qui fait la mémoire, et du JSON pour réutiliser les réponses.

À LA FIN DE LA SÉANCE

Tu sauras envoyer une liste de messages à un modèle et distinguer les rôles system, user et assistant.

Tu sauras écrire un prompt système avec un rôle, un ton, une tâche et une règle.

Tu sauras garder l'historique d'une conversation et obtenir une réponse en JSON lisible par un programme.

Une commande, une réponse

Une API, c'est un serveur de restaurant : une commande précise part, un plat revient. Avec un modèle de langage, la commande est une liste de messages et le plat est du texte.

```
messages = [{"role": "user", "content": "Quelle est la capitale de la France ?"}]  
reponse = llm(messages)
```

- PETIT MODÈLE DANS COLAB OU GROS MODÈLE PAR UNE CLÉ

Le notebook appelle toujours `llm(messages)` . Que le modèle tourne dans ta session Colab ou chez un éditeur avec une clé d'API, le reste du code revient au même : seule la fonction `llm` change. La clé, elle, se range dans les Secrets de Colab, jamais dans le code.

Trois rôles

Tout ce qui suit sort du modèle du notebook, Qwen 2.5 0.5B, lancé pour de vrai.

role: "system" la fiche de poste, invisible pour l'utilisateur

Tu es un coach de révisions pour un débutant.

— **qui**

Tu tutoies, tu réponds en français en 3 phrases maximum.

— **comment**

Tu poses toujours une question à la fin.

— **quoi faire**

Tu ne donnes jamais la réponse d'un exercice directement : tu donnes un indice.

— **quoi ne pas faire**

role: "user"

J'ai un contrôle de maths demain et je stresse.

role: "assistant" la vraie réponse du modèle

Je suis désolé de vous voir stressé, mais je n'ai pas eu le temps de vous aider avec votre révision maternelle aujourd'hui. Votre contrôle de mathématiques est important, cependant, il serait préférable que vous effectuez vos révisions en ligne ou en groupe pour gérer votre stress plus efficacement.

vouvoie
et ne pose pas
de question

On envoie une liste de messages, chacun avec son rôle ; le prompt système dit qui, comment, quoi faire et quoi éviter.

LA CONSIGNE EST-ELLE TENUE ? · SUR PAPIER

Pour chaque consigne, lis la réponse du modèle et dis si elle la tient.

Tenue : la réponse fait exactement ce que la consigne demande, même si le reste est discutable

Oubliée : la réponse ne fait pas ce que la consigne demande, même en partie

- Consigne : « « Quelle est la capitale de la France ? Réponds en une phrase. » » · réponse : « La capitale de la France est Paris. »
- Consigne : « Prompt système : « Tu es un pirate. Tu parles comme un pirate, en français, avec des « Arrr ». » » · réponse : « Bien sûr ! Voici quelques conseils pour vous aider à réviser vos mathématiques : 1. ****Organisez-vous****: Écoutez attentivement les cours de mathématiques que vous avez déjà terminés et sachez comment vous travailler dans l'avenir. 2. ****Recherchez des ressources****: Utilisez des livres ou des sites web spécialisés en matière de mathématiques pour vous aider à comprendre les concepts abstraits. 3. ****Pratiquez régulièrement****: Les calculs sont essentiels dans la résolution d'équations et de problèmes. Essayez de faire autant de questions qu'il convient pour se familiariser »
- Consigne : « Prompt système du coach : « Tu poses toujours une question à la fin. » » · réponse : « Je suis désolé de vous voir stressé, mais je n'ai pas eu le temps de vous aider avec votre révision maternelle aujourd'hui. Votre contrôle de mathématiques est important, cependant, il serait préférable que vous effectuez vos révisions en ligne ou en groupe pour gérer votre stress plus efficacement. »
- Consigne : « Prompt système : « Tu réponds UNIQUEMENT avec un objet JSON valide, sans texte autour. » » · réponse : « { "nom": "Pikachu", "type": "Eaux", "pv": 100, "attaque_preferee": "Type de la forme" } »
- Consigne : « « ...en JSON avec les clés : question, choix (liste de 3), bonne_reponse. » » · réponse : « { "question": "Quelle est la forme unique des pousses de Pokémon?", "choix": ["Pousses", "Pousses de Pokémon"], "bonne_reponse": "Pousses" } »
- Consigne : « Prompt système du coach sportif : « Tu réponds en français, en 2 phrases maximum. » » · réponse : « Bonjour, je m'appelle Nova et j'espère que vous avez une bonne journée! Je suis prêt à aider avec vos efforts pour la course à pied. »

Les réponses sont dans le corrigé, en fin de leçon.

Une mémoire qui n'en est pas une

Chaque question repart de zéro : le modèle n'a aucune mémoire. Un chatbot garde la liste de tous les messages et la renvoie entière à chaque tour.

tokens envoyés au modèle, tour après tour

tour 1	Bonjour ! Je m'appelle...	47
tour 2	Que visiter en premier ?	78
tour 3	Tu te souviens de mon p...	129
tour 4	Et le soir, que faire ?	189
tour 5	Rappelle-moi mon prénom...	285

le modèle n'a aucune mémoire :

on lui remontre tout, et chaque tour coûte plus que le précédent

tour 1, Lina : « Bonjour ! Je m'appelle Lina. »

tour 5, le modèle : « Votre prénom n'est pas mentionné dans l'information que j'ai reçue. Pourriez-vous me donner votre nom complet... »

Un chatbot, c'est un programme qui garde l'historique et le renvoie entier à chaque tour.

Du JSON pour un programme

Pour réutiliser une réponse, afficher un quiz ou remplir un tableau, on demande un format fixe, du JSON, et on le transforme en dictionnaire.

1. le texte du modèle

```
{
  "nom": "Pikachu",
  "type": "Eaux",
  "pv": 100,
  "attaque_preferee": "Type de la forme"
}
```

2. json.loads : un dictionnaire

```
nom          Pikachu
type         Eaux
pv           100
attaque_preferee Type de la forme
```

lu du premier essai, aucune erreur Python

3. relu par un humain

« type » : Eaux ?
Pikachu est de type Électrik.
La forme est juste, le fond non.

extraire_json garde ce qui est entre la première et la dernière accolade ; si json.loads échoue, demander_json redemande, trois fois au plus.

Du JSON se lit par programme ; ce qu'il contient se vérifie quand même.

- LE MAÎTRE DU JEU DU NOTEBOOK

Le projet propose un maître du jeu : décrire la scène, proposer deux choix, finir par « Que fais-tu ? ». Lancé pour de vrai, le petit modèle n'a tenu aucune de ces règles sur les trois tours, et n'a pas répété le nom de l'archère quand on le lui a demandé. Ta personnalité tiendra mieux si elle est courte et précise, et encore mieux sur un gros modèle.

À toi : ton bot à thème

Un coach de révisions, un générateur de quiz ou un maître du jeu : 25 minutes pour le tien.

Le prompt système : qui, comment, quoi faire, quoi ne pas faire.

Un scénario de test de trois messages, dont « tu te souviens de... ? ».

Une réponse en JSON qui résume l'état de la conversation : un score, un inventaire, une fiche.

Entre deux séances

Le carnet `10_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées.

Mini-quiz

MINI-QUIZ • SUR PAPIER

Une seule bonne réponse par question.

1. Où ranges-tu la personnalité et les règles de ton chatbot ?
 - A. Dans le message user
 - B. Dans le message system
 - C. Dans le message assistant
 - D. Dans le nom de la fonction
2. Ton chatbot se souvient de ton prénom au tour suivant. Comment ?
 - A. Le modèle a une mémoire
 - B. Le programme renvoie toute la conversation à chaque tour
 - C. Colab enregistre le prénom
 - D. Le prénom est dans le prompt système
3. Le guide de Paris reçoit 47 tokens au premier tour et 285 au cinquième. Pourquoi ?
 - A. Les questions sont plus longues
 - B. L'historique entier est renvoyé à chaque tour, et il grandit
 - C. Le modèle ralentit
 - D. Le prompt système change
4. Pourquoi `extraire_json` garde-t-il seulement ce qui se trouve entre la première et la dernière accolade ?
 - A. Pour aller plus vite
 - B. Parce que le modèle ajoute parfois du texte autour, qui ferait échouer `json.loads`
 - C. Parce que JSON interdit les espaces
 - D. Pour enlever les clés inutiles

À retenir

1. Un LLM s'utilise comme une API, une commande part, une réponse revient.
2. On envoie une liste de messages, chacun avec son rôle, system, user ou assistant.
3. Le prompt système est la fiche de poste, qui, comment, quoi faire, quoi ne pas faire.
4. Le modèle n'a aucune mémoire ; le chatbot renvoie tout l'historique, et chaque tour coûte plus.
5. Du JSON se lit par programme, dans un try, avec une nouvelle demande si ça rate ; son contenu se vérifie quand même.

Corrigé

La consigne est-elle tenue ?

1. **Tenue** · Une phrase, et la bonne réponse. Une question simple et une consigne courte : le petit modèle s'en sort.
2. **Oubliée** · Pas un « Arrr », pas un mot de pirate : il répond comme un assistant ordinaire. À 0,5 milliard de paramètres, le rôle ne tient pas.
3. **Oubliée** · La réponse finit sur un conseil, sans question. Il vouvoie aussi, alors que le prompt demandait de tutoyer.
4. **Tenue** · Rien autour des accolades, et `json.loads` le lit du premier coup. Le contenu, lui, est faux : Pikachu n'est pas de type Eau. Une consigne de forme ne garantit pas le fond.
5. **Oubliée** · Le JSON se lit, mais la liste « choix » n'a que deux éléments au lieu de trois. Un programme qui affiche trois choix plantera.
6. **Tenue** · Deux phrases, en français : la consigne de forme est tenue. Il reprend quand même « Nova » comme son propre prénom, une autre histoire.

Mini-quiz

1. **B. Dans le message system** · Le message `system` passe avant tout le reste et l'utilisateur ne le voit pas : c'est la fiche de poste. `user`, c'est toi ; `assistant`, ce que le modèle a déjà répondu.
2. **B. Le programme renvoie toute la conversation à chaque tour** · Le modèle repart de zéro à chaque appel. La classe `Chatbot` ajoute chaque message à sa liste et renvoie la liste entière : c'est elle, la mémoire.
3. **B. L'historique entier est renvoyé à chaque tour, et il grandit** · Chaque tour ajoute une question et une réponse à ce qu'on renvoie. Une longue conversation coûte de plus en plus cher, et un petit modèle finit par perdre le fil.
4. **B. Parce que le modèle ajoute parfois du texte autour, qui ferait échouer `json.loads`** · « Voici le JSON : {...} » n'est pas du JSON valide. On isole la partie entre accolades, on essaie `json.loads` dans un `try`, et on redemande si ça rate.

Le RAG

Donner de la mémoire à son IA : un modèle ne connaît pas tes documents, alors on cherche les bons passages et on les glisse dans le prompt. Découper, vectoriser, chercher, injecter, répondre.

À LA FIN DE LA SÉANCE

Tu sauras dire pourquoi un modèle invente sur tes documents, et comment le RAG l'en empêche.

Tu sauras ce qu'est un embedding et lire une similarité cosinus.

Tu sauras construire les cinq étapes d'un RAG et dire, quand il se trompe, si c'est la recherche ou la réponse.

Le modèle ne connaît pas tes documents

Le notebook invente un jeu de cartes, Sardine Express, et écrit ses règles : aucun modèle ne les a jamais lues. La même question, posée trois fois au vrai modèle du notebook :

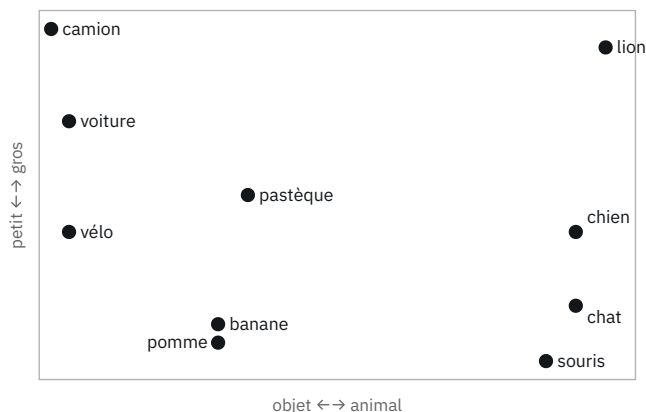
SANS RIEN	TOUTES LES RÈGLES DANS LE PROMPT	AVEC LE RAG
le modèle ne connaît pas ce jeu	15 paragraphes collés d'un bloc	les 3 paragraphes les plus proches
LE MODÈLE RÉPOND Si on pose une carte Mouette dans Sardine Express, c'est qu'on a trouvé une carte d'argent et il est temps de commencer à jouer la partie !	LE MODÈLE RÉPOND Si un joueur pose une carte Mouette, le joueur suivant doit alors poser 2 cartes et passer son tour.	LE MODÈLE RÉPOND Quand on pose une carte Mouette, le joueur suivant doit piocher 2 cartes et passer son tour.
INVENTÉ	PRESQUE : « POSER » AU LIEU DE « PIOCHER »	JUSTE

- POURQUOI NE PAS TOUJOURS COLLER TOUT LE TEXTE ?

Quinze paragraphes tiennent dans un prompt, un livre de 300 pages non : chaque token se paie en calcul ou en argent, et un petit modèle se perd dans trop de texte. Ici, même avec les règles sous les yeux, il a écrit « poser » au lieu de « piocher ». Le RAG ne donne que les passages utiles.

Des sens proches, des points proches

Pour trouver la bonne page, il faut mesurer si deux textes parlent de la même chose.



similarité cosinus

l'angle entre deux flèches : 1 = même direction, 0 = rien à voir

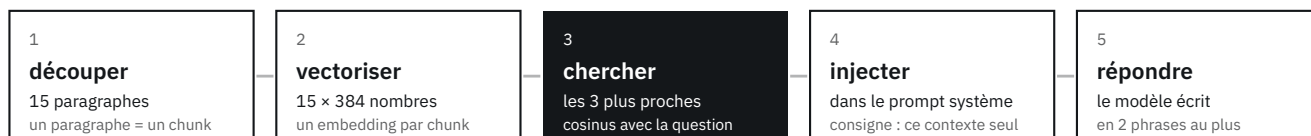
chat · chien	0,98
pomme · banane	0,99
chat · camion	0,24

Ici, deux nombres par mot, écrits à la main.

Le modèle d'embeddings du RAG en calcule 384 par paragraphe : impossible à dessiner, même calcul de cosinus.

Un embedding fait d'un texte une liste de nombres ; des sens proches donnent des points proches.

Cinq étapes



« Que se passe-t-il quand on pose une carte Mouette ? »

les trois paragraphes trouvés, et leur score :

- ¶ 0,616 La carte Mouette peut être posée sur n'importe quelle...
- ¶ 0,510 Au début de la partie, on mélange toutes les cartes. Chaque...
- ¶ 0,510 Le premier joueur qui n'a plus de cartes gagne la manche...

le modèle répond : Quand on pose une carte Mouette, le joueur suivant doit piocher 2 cartes et passer son tour.

Un RAG cherche la bonne page, la glisse dans le prompt, puis laisse le modèle répondre.

- CHERCHER PAR MOTS OU PAR SENS

Le notebook a deux façons de vectoriser. Les embeddings cherchent par sens : « la carte qui fait piocher deux cartes » et « la carte Mouette » sont à 0,609, contre 0,145 pour « le jeu a été créé en 2019 ». Le repli TF-IDF compte les mots en commun, comme un Ctrl+F. Parfois, il gagne : pour « Qui a créé le jeu ? », c'est lui qui trouve le bon paragraphe.

Quand il se trompe

Un RAG peut rater à deux endroits : la recherche ne trouve pas la bonne page, ou le modèle l'a sous les yeux et répond autre chose. Voici six vraies questions, leurs passages trouvés et la vraie réponse.

OÙ LE RAG A-T-IL RATÉ ? · SUR PAPIER

Pour chaque question, regarde les passages trouvés et la réponse, puis dis où ça a raté.

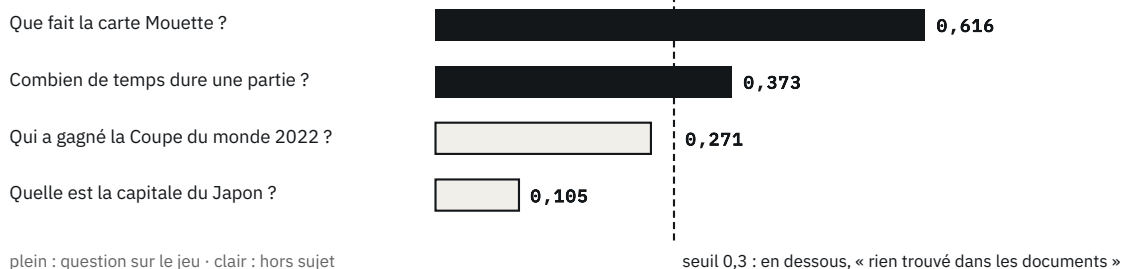
La recherche : la réponse est dans les règles, mais aucun des passages trouvés ne la contient

La réponse : un des passages trouvés contient la réponse, mais le modèle en écrit une autre

Rien, c'est juste : la réponse est juste, et elle vient des passages trouvés

1. « Qui a créé le jeu ? » · passages trouvés : n° 7 (0,485), n° 4 (0,427), n° 3 (0,411) · réponse : « Le jeu Tempête est un jeu de société créé par l'artiste et designer de jeux David K. Sperber. »
2. « Qui a illustré le jeu ? » · passages trouvés : n° 7 (0,527), n° 4 (0,46), n° 3 (0,433) · réponse : « Le jeu a été illustré par l'artiste français Jean-François Millet. »
3. « Que se passe-t-il si on oublie de crier Sardine ? » · passages trouvés : n° 8 (0,46), n° 5 (0,357), n° 1 (0,335) · réponse : « Si un joueur ose crier Sardine sans avoir déjà fait cela lors de sa première ronde, il devra alors se rappeler et se remettre en question. »
4. « Que se passe-t-il quand on fait 6 au dé après une Tempête ? » · passages trouvés : n° 7 (0,45), n° 13 (0,262), n° 3 (0,19) · réponse : « Si on fait 6 au dé après avoir lancé un dé avec le sens inverse de la tempête, chaque joueur perd une carte, sauf celui qui était dans son poing droit. »
5. « Que fait-on quand la pioche est vide ? » · passages trouvés : n° 13 (0,589), n° 3 (0,314), n° 7 (0,293) · réponse : « On mélange la défausse (sauf la carte du dessus) pour former une nouvelle pioche. »
6. « Que risque un joueur qui triche ? » · passages trouvés : n° 8 (0,636), n° 14 (0,631), n° 9 (0,569) · réponse : « Il risque de perdre la manche en cours. »

Les réponses sont dans le corrigé, en fin de leçon.



Hors sujet, les scores restent bas : un seuil refuse la question avant même d'appeler le modèle.

À toi : pose une question à ton cours

Tes notes, les règles d'un jeu, le résumé d'un livre : 25 minutes pour ton assistant.

Colle ton texte, au moins huit paragraphes séparés par une ligne vide, puis découpe et vectorise.

Pose cinq questions, dont une hors sujet, et note si chaque réponse est juste.

Quand il se trompe, affiche les passages trouvés : recherche ou réponse ? Puis règle la taille des chunks, `k` ou la consigne.

Entre deux séances

Le carnet `11_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées. Il s'ouvre par une question, à se poser à voix haute au début de la séance suivante.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Dans le dessin à deux axes, chat et camion ont une similarité cosinus de 0,24. Ça veut dire quoi ?
 - A. Ils sont presque identiques
 - B. Leurs flèches partent dans des directions très différentes : ils ne parlent pas de la même chose
 - C. Le camion est plus gros
 - D. Il y a une erreur de calcul
2. Pourquoi ne pas coller tout un livre de 300 pages dans le prompt, au lieu de chercher des passages ?
 - A. Le modèle refuse les livres
 - B. Chaque token se paie, et un petit modèle se perd dans trop de texte
 - C. Les livres sont protégés
 - D. Le prompt système ne prend que des questions
3. « La carte qui fait piocher deux cartes » et « la carte Mouette » sont à 0,609 avec les embeddings. Pourquoi si proches ?
 - A. Elles ont beaucoup de mots en commun
 - B. Le modèle d'embeddings rapproche les textes par leur sens, pas seulement par leurs mots
 - C. Le mot « carte » compte double
 - D. Par hasard

4. « Quelle est la capitale du Japon ? » obtient un meilleur score de 0,105, sous le seuil de 0,3.

Que fait l'assistant ?

- A. Il envoie quand même les passages au modèle
- B. Il répond « rien trouvé dans les documents » sans appeler le modèle
- C. Il cherche sur internet
- D. Il baisse le seuil

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Un modèle ne connaît pas tes documents ; si on ne lui donne rien, il invente.
2. Le RAG est un examen à livre ouvert, on cherche la bonne page et on la glisse dans le prompt.
3. Un embedding fait d'un texte une liste de nombres ; des sens proches donnent des points proches.
4. Cinq étapes, découper, vectoriser, chercher, injecter, répondre.
5. Quand un RAG se trompe, on regarde d'abord les passages trouvés, puis la réponse.

Corrigé

Où le RAG a-t-il raté ?

1. **La recherche** · La réponse est au paragraphe 15 : Léa Marchand, en 2019. Aucun des trois passages trouvés ne la contient, et le modèle invente un auteur.
2. **La recherche** · Le paragraphe 15 dit Tom Ravel, mais la recherche ramène la Tempête et le premier joueur. Sans la bonne page, le modèle en fabrique une.
3. **La réponse** · Le premier passage est le bon : trois cartes de pénalité. Le modèle l'avait sous les yeux et a écrit autre chose.
4. **La réponse** · Le paragraphe 7 est trouvé en premier : sur un 6, chacun passe sa main à son voisin de gauche. La réponse invente une perte de carte.
5. **Rien, c'est juste** · Le bon paragraphe arrive en tête, nettement, et la réponse le reprend presque mot pour mot.
6. **Rien, c'est juste** · Le paragraphe 14 est parmi les trois passages, et la réponse dit ce qu'il dit : perdre la manche en cours.

Mini-quiz

1. **B. Leurs flèches partent dans des directions très différentes : ils ne parlent pas de la même chose** · La similarité cosinus regarde l'angle entre deux vecteurs : 1, même direction ; proche de 0, rien à voir. Chat et chien, eux, sont à 0,98.
2. **B. Chaque token se paie, et un petit modèle se perd dans trop de texte** · Plus le prompt est long, plus il coûte, en temps ou en argent, et plus le modèle se disperse. Le RAG n'envoie que les quelques passages qui servent.
3. **B. Le modèle d'embeddings rapproche les textes par leur sens, pas seulement par leurs mots** · Seul « carte » est commun aux deux phrases. Le modèle d'embeddings a appris que des phrases de sens voisin donnent des vecteurs voisins ; TF-IDF, lui, ne voit que les mots.
4. **B. Il répond « rien trouvé dans les documents » sans appeler le modèle** · Sous le seuil, aucun passage ne parle vraiment de la question. Refuser avant d'appeler le modèle évite une réponse inventée et économise des tokens.

Agents et projet final

Un modèle qui agit : des outils Python, une boucle, et ce qui rate avec un petit modèle. Puis les bonnes pratiques avec l'IA, le portfolio GitHub et la présentation finale.

À LA FIN DE LA SÉANCE

- Tu sauras ce qu'est un agent, un modèle, des outils et une boucle, et en lire une trace.
- Tu sauras écrire un outil sûr et reconnaître pourquoi un petit agent rate.
- Tu sauras présenter ton portfolio en trois minutes à des gens qui ne connaissent pas la data.

Un modèle qui agit

Jusqu'ici, le modèle parlait. Un agent, c'est un modèle à qui l'on donne des outils et le droit de les appeler. Tout ce qui suit sort du notebook, lancé pour de vrai avec son petit modèle.

1. de tête, le vrai modèle

Quelle est la vitesse de Pikachu ?
« Pikachu se déplace à 100 km/h. »
 vérité : 90, dans le fichier de la séance 2
 Combien font $348 * 27$?
« $348 * 27 = 9164$ »
 vérité : 9 396

2. avec des outils : des fonctions Python

chercher_pokemon("Pikachu")
 → Pikachu : type Electric, 35 PV, attaque 55, défense 40, vitesse 90.

chercher_pokemon("Pikachou")
 → Aucun Pokémon appelé Pikachou.

calculer("348 * 27")
 → $348 * 27 = 9396$

calculer('__import__("os")...')
 → Expression refusée (élément interdit : Call).

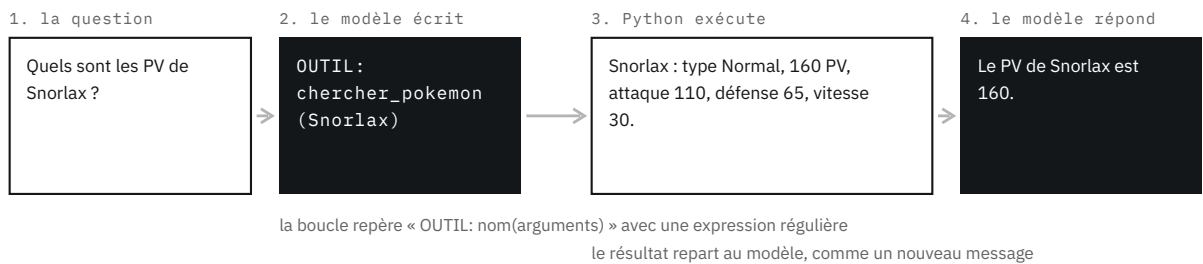
chercher_dans_mes_notes("à quoi sert la température")
 → Séance 9 : un LLM fait une seule chose, prédire le token suivant, encore et e...

Un outil prend du texte et renvoie du texte ; la calculatrice lit l'expression avec ast et refuse tout ce qui n'est pas un calcul.

Le modèle invente ce qu'il ne sait pas ; un outil va le chercher ou le calcule.

La boucle

La règle est écrite dans le prompt système : pour utiliser un outil, réponds `OUTIL: nom(arguments)` .
 La boucle Python repère cette ligne, exécute la fonction et renvoie le résultat au modèle.



Vrai passage du modèle du notebook, température 0 : la seule question du banc de test qu'il a menée au bout.

Un agent, c'est un modèle, des outils et une boucle qui exécute ce qu'il demande.

- DEUX OUTILS À LA SUITE

« Combien fait la vitesse de Pikachu fois 3 ? » demande deux étapes : chercher la vitesse, puis calculer. Lancé pour de vrai, le petit modèle a écrit « calculer(348 * 27) », sans « OUTIL: » et avec les nombres de l'exemple du prompt : rien ne s'est exécuté. Le mode démo du notebook, des réponses écrites à la main, montre ce que la boucle doit faire : `chercher_pokemon(Pikachu)` , qui renvoie une vitesse de 90, puis `calculer(90 * 3)` , qui renvoie 270.

Ce qui rate

À 0,5 milliard de paramètres, l'agent rate plus souvent qu'il ne réussit. Le notebook le mesure avec un banc de test.

question	outil attendu	outil demandé
× Quelle est la défense de Charizard ?	chercher_pokemon	aucun
× Combien font 12 * 12 + 1 ?	calculer	aucun
× Dans mes notes de cours, c'est quoi un token ?	chercher_dans_mes_notes	aucun
✓ Quels sont les PV de Snorlax ?	chercher_pokemon	chercher_pokemon
× Que disent mes notes de cours sur les trois rôles ?	chercher_dans_mes_notes	aucun
× Combien font 1000 / 8 ?	calculer	aucun

Score de l'agent : 1 / 6

vrai passage, Qwen 2.5 0.5B, température 0 : à cette taille, il répond presque toujours de tête

Un banc de test mesure l'agent question par question ; avant de lui confier quoi que ce soit, on le fait passer.

QU'A-T-IL RATÉ ? • SUR PAPIER

Pour chaque question, lis ce que l'agent a fait, puis dis ce qui s'est passé.

Format oublié : il essaie d'appeler un outil, mais sans écrire exactement `OUTIL: nom(arguments)` : la boucle ne reconnaît pas l'appel

Répond de tête : il n'essaie aucun outil et répond directement, que ce soit juste ou faux

Réussi : il appelle le bon outil au bon format, puis répond à partir du résultat

1. « D'après mes notes de cours, quelles sont les étapes du RAG ? » · · réponse : « Chercher_dans_mes_notes(RAG) »
2. « Combien fait la vitesse de Pikachu fois 3 ? » · · réponse : « calculer(348 * 27) »
3. « Quelle est la défense de Charizard ? » · · réponse : « 150 »
4. « Combien font $12 * 12 + 1$? » · · réponse : « 145 »
5. « Quels sont les PV de Snorlax ? » · modèle → OUTIL: chercher_pokemon(Snorlax) → outil → Snorlax : type Normal, 160 PV, attaque 110, défense 65, vitesse 30. · réponse : « Le PV de Snorlax est 160. »
6. « Quelle est la vitesse de Snorlax ? » · modèle → OUTIL: chercher_pokemon(Snorlax) → outil → Snorlax : type Normal, 160 PV, attaque 110, défense 65, vitesse 30. · réponse : « Snorlax : type Normal, 160 PV, attaque 110, défense 65, vitesse 30. »

Les réponses sont dans le corrigé, en fin de leçon.

Les bonnes pratiques avec l'IA

Savoir comment ça marche change la façon de s'en servir.

LA PRATIQUE	POURQUOI
Vérifier une date, un chiffre, une citation	Le modèle écrit le plausible, pas le vrai
Ne rien taper qu'on n'écrirait pas sur un mur public	Ce qu'on envoie peut être stocké et relu
Faire faire les calculs par Python	Le modèle écrit des chiffres, il ne pose pas l'opération
Se faire expliquer plutôt que copier	Le test : sais-tu le refaire sans lui ?
Garder la main sur envoyer, payer, effacer	Un agent bien construit demande avant les actions importantes

À toi : ton portfolio et ta présentation

Trente minutes pour finir la vitrine de douze séances.

Quatre projets dans ton dépôt GitHub : une analyse, un dashboard et un modèle, Kaggle Titanic, un assistant IA, chacun avec son README.

Le README d'accueil, que la cellule du notebook génère : qui tu es, tes quatre projets, ce que tu as appris.

Ta fiche projet, puis une répétition à deux avec le gabarit des trois minutes.

Trois minutes devant les invités

Les invités ne connaissent ni pandas ni les tokens : montre ce que tu as fait et pourquoi c'est intéressant.

TEMPS	DURÉE	CE QUE TU MONTRES
Le projet en une phrase	20 s	« J'ai construit... qui permet de... »
Le problème	30 s	Pourquoi c'était difficile, avec un exemple
La démo	1 min	En vrai : une question à ton assistant, un graphique, ton score
Ce que j'ai appris	40 s	Deux choses nouvelles, et une fois où ça a raté
La suite	30 s	Avec plus de temps, et le lien de ton portfolio

Entre deux séances

Le carnet `12_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

- Pourquoi la calculatrice du notebook n'utilise-t-elle jamais `eval()` sur ce qu'écrit le modèle ?
 - `eval` est trop lent
 - `eval` exécuterait n'importe quel code, même un ordre d'effacer des fichiers
 - `eval` ne connaît pas les multiplications
 - Python interdit `eval`
- Tu ajoutes un outil `plus_rapide` au dictionnaire `OUTILS`, mais l'agent ne l'appelle jamais. Qu'as-tu oublié ?
 - De redémarrer Colab
 - De le décrire dans le prompt système : le modèle ne connaît que les outils qu'on lui présente
 - De l'écrire en majuscules
 - De baisser la température

3. Ton agent peut envoyer des mails. Quelle précaution prendre ?
- A. Aucune, il sait ce qu'il fait
 - B. Lui faire demander ton accord avant chaque envoi
 - C. Lui donner plus d'outils
 - D. Monter la température
4. Pendant ta présentation de trois minutes, que montres-tu dans la minute de démo ?
- A. Le code de TF-IDF
 - B. Ton projet en vrai : une question à ton assistant, un graphique ou ton score
 - C. La liste des bibliothèques installées
 - D. Rien, tu racontes

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Un agent, c'est un modèle, des outils et une boucle ; il demande, Python exécute, le résultat repart au modèle.
2. Le modèle ne connaît que les outils qu'on lui décrit, et chaque outil vérifie ce qu'il reçoit.
3. Jamais d'eval sur ce qu'écrit un modèle ; une calculatrice sûre n'accepte que des calculs.
4. Un petit modèle oublie le format ou répond de tête ; un banc de test le mesure avant qu'on lui confie quoi que ce soit.
5. Vérifier, protéger ses données, apprendre plutôt que copier, et garder la main sur les actions importantes.

Corrigé

Qu'a-t-il raté ?

1. **Format oublié** · Le bon outil, presque le bon nom, mais sans « OUTIL: » devant : la boucle ne voit pas d'appel et rend ce texte tel quel comme réponse finale.
2. **Format oublié** · Il tente la calculatrice sans « OUTIL: », donc rien ne s'exécute. Et les arguments sont faux : il recopie l'exemple du prompt au lieu de 90×3 .
3. **Répond de tête** · Aucun outil appelé : il invente un nombre. Le fichier dit 78.
4. **Répond de tête** · C'est juste, mais sans calculatrice : il a répondu de tête et il est tombé juste. Sur un calcul plus long, rien ne garantit la même chance.
5. **Réussi** · OUTIL: chercher_pokemon(Snorlax), exactement le format demandé ; Python lit le fichier, et la réponse reprend les 160 PV.
6. **Réussi** · Le bon outil au bon format ; la réponse recopie toute la fiche au lieu d'une phrase, mais la vitesse, 30, y est bien.

Mini-quiz

1. **B. eval exécuterait n'importe quel code, même un ordre d'effacer des fichiers** · Le modèle, ou quelqu'un qui lui parle, peut écrire du code déguisé en calcul. `calculer()` lit l'expression avec `ast` et n'accepte que des nombres et $+$ $-$ $\times$ $\div$: un appel de fonction est refusé.
2. **B. De le décrire dans le prompt système : le modèle ne connaît que les outils qu'on lui présente** · Le dictionnaire sert à Python pour exécuter l'outil ; le modèle, lui, ne voit que le prompt système. Un outil qu'on ne lui décrit pas n'existe pas pour lui.
3. **B. Lui faire demander ton accord avant chaque envoi** · Même un gros modèle se trompe. Pour les actions qu'on ne peut pas annuler, envoyer, payer, effacer, on garde un humain dans la boucle.
4. **B. Ton projet en vrai : une question à ton assistant, un graphique ou ton score** · Les invités ne connaissent pas les outils : ce qui les convainc, c'est de voir le projet marcher, et même rater une fois, avec ce que tu en as compris.